

Universidad de La Habana
Facultad de Matemática y Computación



Un generador de problemas prueba para evaluar la calidad de la solución de los algoritmos de problemas de optimización de dos niveles

Autor:

Francisco Vicente Suárez Bellón

Tutora:

Dr. C. Gemayqzel Bouza Allende

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

10 de febrero de 2025

github.com/FVSB/Tesis

“En matemáticas no entiendes las cosas. Solo te acostumbras a ellas.”
— **John von Neumann**

Agradecimientos

El camino recorrido hasta este momento ha sido posible gracias al apoyo y dedicación de personas extraordinarias que han estado presentes en cada paso de mi formación académica. Deseo expresar mi más sincero agradecimiento a todos los profesores que han contribuido a mi desarrollo profesional durante mi trayectoria universitaria.

Mi más profunda gratitud a la profesora Dra. C. Gemayqzel, mi tutora, por su paciencia, dedicación y capacidad para transmitir conocimientos en el fascinante mundo de la optimización binivel. Su apoyo y comprensión han sido fundamentales para la culminación exitosa de este trabajo.

A mi madre, mi mayor inspiración, mi héroe y mi pilar inquebrantable, le debo el más grande de los agradecimientos. Por ser madre y padre, por estar presente en los momentos más difíciles y por anteponer siempre mis sueños a los suyos. Gracias por tu amor incondicional, por tus sacrificios y por ser ese refugio al que siempre puedo acudir. Eres el ejemplo más claro de fortaleza y entrega, y todo lo que soy te lo debo a ti.

A mi padre, aunque físicamente ya no está conmigo, su legado de resiliencia y espíritu emprendedor sigue vivo en mí. Gracias por enseñarme, a través de tu ejemplo, que no hay obstáculo que no pueda superarse con esfuerzo y determinación. Tu presencia se siente en cada logro que alcanzo, y sé que desde algún lugar estás orgulloso de mí.

A mis abuelos, Ada y Vicente, quienes han sido como unos segundos padres para mí. Gracias por su amor incondicional, por su apoyo constante y por ser un ejemplo de vida. A mi abuela, por ser ese refugio de amor y consuelo en los momentos más oscuros, y por tener siempre una palabra de aliento y un plato de comida caliente después de los días más difíciles. A mi abuelo, por ser mi guía desde niño, por involucrarse activamente en mi educación

y por anteponer siempre mi felicidad a la suya. Su presencia en mi vida ha sido un regalo invaluable.

A mis tíos, Rozana y Alejandro, quienes han sido como unos padres para mí. Gracias por su apoyo incondicional, por sus consejos sabios y por enseñarme el valor del esfuerzo y la perseverancia. A mi tía, por su amor infinito y por estar siempre en los momentos más difíciles. A mi tío, por ser un amigo que, incluso en las discusiones, siempre me guía por el camino correcto. Su cariño y dedicación han sido un pilar fundamental en mi vida.

A mi prima Diana, mi cómplice y mi guía, quiero agradecerte por tener paciencia y confianza con aquel niño travieso y complicado. Gracias por esas noches enteras explicándome el teorema de Kronecker-Capelli, por tu apoyo incondicional y por ser esa amiga leal en quien siempre puedo confiar. Tu generosidad y tu amor han sido un faro en mi camino.

A mis bisabuelos, Marta y Rubén, quienes con su dulzura y sentido del humor han llenado de luz los días más grises. Gracias por su amor incondicional y por ser el alma de nuestra familia. Su presencia ha sido un regalo que atesoro con todo mi corazón.

A Margarita y Roberto, quienes han sido como unos abuelos para mí, les agradezco por su apoyo constante y por estar siempre presentes en los momentos más difíciles. Su cariño y su compañía han sido un gran consuelo en este proceso.

A mis fieles compañeros de cuatro patas, Tyrion y Valentina, gracias por su amor incondicional y por ser ese refugio de alegría en los días de mayor estrés.

A mi familia, quiero agradecerles especialmente por su infinita paciencia y dedicación en los momentos más difíciles de este proceso. Quiero darle las gracias por su apoyo constante y su aliento para no abandonar nunca, lo que me ha motivado a seguir adelante y alcanzar mis metas.

A mis compañeros más cercanos: Carla, Alfredo, Karen y Héctor. A Carla, por ser esa compañera incansable en las noches de entregas a las 11:59, los momentos de sabiduría, risas, llanto, apoyo, comprensión y por ese poquito que ni el Álgebra ni la Lógica saben explicar, en fin por esos momentos brillantes que hemos compartido. A Alfredo, por su peculiar

manera de hacerme reír incluso en los momentos más estresantes (sobre todo si el código no tiene docstring) y su apoyo incondicional. Y a Karen y Héctor, por su amistad inquebrantable y por estar siempre ahí cuando más los necesito.

A mis amigos y compañeros de estudio, gracias por su tiempo, dedicación y palabras de aliento, que enriquecieron esta experiencia académica.

Y, por supuesto, a ChatGPT, gracias por ser una herramienta invaluable en la redacción y organización de este trabajo. Tu ayuda ha sido fundamental para dar forma a mis ideas y expresarlas de manera clara y coherente.

Este trabajo no habría sido posible sin el soporte y la guía de cada uno de ustedes. Gracias por ser parte fundamental de este capítulo de mi vida.

Opinión del tutor

La optimización binivel es, efectivamente, un problema complejo. La clave es la dependencia jerárquica: una decisión en el nivel superior afecta las opciones disponibles en el nivel inferior, y la solución óptima del nivel inferior a su vez impacta la función objetivo del nivel superior. Esto crea un ciclo de retroalimentación que dificulta la búsqueda de la solución óptima global. La estrategia de transformar el problema binivel a un MPEC (Mathematical Program with Equilibrium Constraints o Problema de Programación Matemática con Restricciones de Equilibrio) es una técnica común. Estas restricciones de complementariedad son las que hacen que los MPEC sean difíciles de resolver. La Contribución esta tesis es la generación de problemas cuya transformación MPEC tiene un punto estacionario con ciertas propiedades importantes en desde un punto de vista teórico y algorítmico. Tener una forma de generar problemas con puntos estacionarios de cierta clase podría ser muy valioso para evaluar el rendimiento de diferentes algoritmos de resolución de MPECs en problemas con características específicas, comprender las propiedades de estos modelos explorando la relación entre la estructura del problema y las propiedades de sus soluciones estacionarias. En La capacidad de generar problemas MPEC con tipos específicos de puntos estacionarios es una innovación que podría ser muy útil para la comunidad investigadora. Cabe destacar que esto se logra adicionando funciones lineales con lo que propiedades como linealidad y convexidad de las funciones involucradas no se ven afectadas. Para un estudiante de Computación desarrollar esta tesis no es simple. Tuvo que adentrarse en el difícil campo de la teoría de la optimización binivel para lograr una programación del generador, utilizar Julia y entender los algoritmos de solución que se aplican. Francisco ha sido trabajador, meticuloso, interesado y ha logrado llevar esta tesis a feliz término con independencia. Ha enfrentado varios retos y ha logrado sortearlos usando lo aprendido. Espero esta experiencia le sea útil en su vida futura

Resumen

El problema de optimización binivel se define como minimizar una función sobre un conjunto determinado por los puntos óptimos de un modelo de programación matemática. La optimización en el nivel inferior depende de las decisiones tomadas en el nivel superior, creando así una relación de interdependencia entre ambos niveles.

Para abordar este problema, se considera la creación de un problema relacionado en el cual el nivel inferior se sustituye por las condiciones necesarias de optimalidad, siendo este un problema con restricciones de complementariedad.

En este trabajo se propone una forma de generar problemas de dos niveles cuyo problema con restricciones de complementariedad (MPEC) tiene un punto estacionario perteneciente a una de las siguientes clases: fuertemente estacionario, M-estacionario o C-estacionario, dependiendo de los multiplicadores.

Palabras clave: Optimización binivel, MPECs, Condiciones necesarias de optimalidad, Punto estacionario.

Abstract

The bilevel optimization problem is defined as minimizing a function on a set determined by the optimal points of a mathematical programming model. The optimization at the lower level depends on the decisions made at the upper level, thus creating an interdependent relationship between both levels.

To address this problem, the lower-level problem is replaced by the necessary optimality conditions and the resulting (relaxed) optimization problem with complementarity constraints is solved.

This work proposes a way to generate two-level problems whose relaxed problem has a stationary point belonging to one of the following classes: strongly stationary, M-stationary, or C-stationary, depending on the multipliers.

Keywords: Bi-level optimization, MPECs, Necessary optimality conditions, Stationary point.

Índice general

Introducción	1
1. Preliminares	6
1.1. Optimización Binivel	7
1.2. Programación Matemática con Restricciones de Equilibrio (MPEC)	9
1.3. Métodos de Reformulación para Optimización Binivel	12
1.3.1. Método Big-M	12
1.3.2. Método SOS1	12
1.3.3. Método ProductMode	13
1.4. Modelación en Julia	14
2. Detalles de Implementación y Experimentos	21
2.1. Notaciones:	21
2.2. Generación del problema sobre Julia	23
2.2.1. Datos de entrada	23
2.2.2. Modificación del Problema Original	25
2.3. Implementación Algorítmica y Guía de Usuario	27
2.3.1. Declarar Funciones Objetivo	29
2.3.2. Definición del conjunto de Índices activos	29
2.3.3. Introducir el Punto	32
2.3.4. Generar el Problema	32
2.3.5. Ejemplo completo	32
3. Experimentación	34
3.1. Modelación de la experimentación	34
3.2. Metodología de Generación y Experimentación	37
3.2.1. Problemas Lineales	38
3.2.2. Problemas Cuadráticos	39
3.2.3. Problemas No Convexos	40
3.2.4. Análisis de los resultados	40

Conclusiones	42
Bibliografía	44
Anexos	47
3.3. Tablas de resultados	47

Índice de figuras

1.1. Problema de optimización bajo el enfoque optimista.	7
1.2. Problema de optimización bajo el enfoque pesimista.	8

Índice de tablas

1.1. Comparación de métodos de reformulación	13
3.1. Problemas Seleccionados	35
3.2. Semillas utilizadas en problemas Lineales	36
3.3. Semillas utilizadas en problemas Cuadráticos	36
3.4. Semillas utilizadas en problemas No Convexos	37
3.5. Resultados de los problemas lineales del tipo Alpha-Zero	47
3.6. Resultados de los problemas lineales del tipo C-Estacionario	47
3.7. Resultados de los problemas lineales del tipo M-Estacionario	48
3.8. Resultados de los problemas lineales del tipo Fuertemente-Estacionario	48
3.9. Resultados de los problemas cuadráticos del tipo Alpha-Zero	48
3.10. Resultados de los problemas cuadráticos del tipo C-Estacionario . . .	49
3.11. Resultados de los problemas cuadráticos del tipo M-Estacionario . . .	49
3.12. Resultados de los problemas cuadráticos del tipo Fuertemente-Estacionario	50
3.13. Resultados de los problemas No Convexo del tipo Alpha-Zero	50
3.14. Resultados de los problemas No Convexo del tipo C-Estacionario . . .	50
3.15. Resultados de los problemas No Convexo del tipo M-Estacionario . . .	51
3.16. Resultados de los problemas No Convexo del tipo Fuertemente-Estacionario	51

Ejemplos de código

2.1. Importar el Módulo	27
2.2. Crear el modelo para $\alpha = \mathbf{0}$	28
2.3. Crear el modelo para $\alpha \neq \mathbf{0}$	28
2.4. Referirse al nivel superior	28
2.5. Referirse al nivel inferior	28
2.6. Introducir las variables del nivel superior	28
2.7. Introducir las variables del nivel inferior	29
2.8. Declarar una función objetivo del nivel superior	29
2.9. Declarar una función objetivo del nivel inferior.	29
2.10. Definir el conjunto de índice activo	30
2.11. Introducir restricción del nivel superior	31
2.12. Introducir el punto $(1,1,1,1)$	32
2.13. Generar el problema	32
2.14. Script	32

Introducción

La optimización de dos niveles, un área fundamental en la investigación operativa y la teoría de juegos, presenta desafíos significativos debido a su complejidad inherente. Este tipo de problemas se caracterizan por la interacción entre un líder y un seguidor, donde las decisiones del líder afectan las respuestas del seguidor. Uno de los aspectos más críticos de esta problemática es garantizar la existencia de soluciones óptimas, lo cual se ve complicado por la naturaleza no convexa del problema, incluso cuando las funciones y los conjuntos factibles son convexos. A menudo, los algoritmos utilizados en este contexto solo logran identificar puntos estacionarios o críticos, que no necesariamente representan soluciones locales o globales, ver Dempe y Zemkoho 2020a.

El modelo de dos niveles es:

$$\begin{aligned} & \min_x \quad F(x, y) \\ & \text{s.a.} \quad \begin{cases} x \in \mathcal{T}, \\ y \in S(x) = \arg \min_y \{f(x, y) \mid y \in \mathcal{H}(x)\}, \\ (x, y) \in \mathcal{M}^0, \end{cases} \end{aligned}$$

donde $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$, $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $\mathcal{T} \subseteq \mathbb{R}^n$, $f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $\mathcal{H}(x) \subseteq \mathbb{R}^m$, para todo $x \in \mathbb{R}^n$ and $\mathcal{M}^0 \subseteq \mathbb{R}^{n+m}$. En otras palabras, el problema de optimización binivel se centra en que el líder (nivel superior) debe tomar decisiones (x) que optimicen su objetivo $F(x, y)$, anticipando que el seguidor (nivel inferior) responderá de manera óptima con respecto a su propio objetivo $f(x, y)$, dado el valor de x elegido por el líder. Esta interacción jerárquica entre ambos niveles añade una gran complejidad al problema en comparación con los problemas de optimización de un solo nivel.

Los problemas de optimización de dos niveles son muy utilizados para modelar y analizar mercados eléctricos complejos, ofreciendo una perspectiva única sobre las interacciones estratégicas entre diversos agentes económicos. En el trabajo de Aussel, Cervinka y Marechal 2016 se desarrolló un modelo innovador que aborda los mercados de electricidad desregulados. Su enfoque se distingue por incorporar restricciones de producción y pérdidas térmicas, lo que permite una modelización más precisa y realista. Mediante el uso de modelos binivel, los investigadores pueden explorar escenarios más complejos y representativos del funcionamiento real de los mercados

energéticos. Otro ejemplo en esta línea se encuentra en Aussel, Bendotti y Pistek 2017, se profundiza en el análisis de mercados de electricidad de pago por oferta, explorando cómo un productor puede ajustar su estrategia considerando las acciones de sus competidores. El estudio destaca la aplicación de conceptos de equilibrio de Nash y técnicas de mejor respuesta, proporcionando una metodología sofisticada para optimizar la participación de un productor en el mercado.

Los modelos binivel también tiene aplicaciones fundamentales en la selección de hiperparámetros en aprendizaje automático, como lo demuestra el trabajo de Dempe y Zemkoho 2020b. El capítulo 6 del libro aborda la optimización de hiperparámetros en problemas de clasificación y regresión, presentando algoritmos innovadores para manejar funciones objetivo no suaves y no convexas. La razón del uso de esta radica en su capacidad para minimizar errores en modelos complejos, mejorando así la precisión general del aprendizaje automático. Además, se implementan algoritmos especializados para abordar problemas no convexas.

La optimización de dos niveles es una herramienta clave en el diseño y operación de redes industriales sostenibles. Ejemplos notables se incluyen en los estudios de Ramos, Boix et al. 2016 donde se optimiza el uso del agua a partir de modelar la situación mediante juegos de múltiples líderes-seguidores en un Parque Ecológico Industrial (PEI), priorizando objetivos ambientales y económicos. Los resultados mostraron que las empresas participantes lograron beneficios significativos con las formulaciones **KKT** (Karush-Kuhn-Tucker) del modelo **MLFG** (Multi-Leader-Follower Game) utilizado. Además, en Ramos, Boix et al. 2016 se destaca la influencia de la estructura del juego en la configuración óptima, sugiriendo la necesidad de un diseño óptimo para cada planta dentro del PEI. Los enfoques **SLMFG** (Single-Leader-Multifollower Game) y **MLSFG** (Multi-Leader-Single-Follower Game) presentan variaciones en el rol de los participantes: en SLMFG, las empresas son seguidoras y la autoridad es líder, mientras que en MLSFG, ocurre lo contrario. Se resalta que el enfoque MLFG logra equilibrar objetivos económicos y ambientales, generando ahorros significativos mediante la reutilización de recursos. Los resultados indican una reducción en el consumo de agua fresca gracias a las estrategias implementadas, utilizando herramientas como GAMS para modelar los problemas de optimización, ver Ramos, Boix et al. 2016. Además, en Ramos, Rocafull et al. 2018 los autores introducen el concepto de autoridad ambiental en el diseño de redes de servicios públicos, utilizando juegos de múltiples líderes-seguidores y reformulaciones KKT. En el ámbito del despacho energético bajo restricciones de carbono, en Gu et al. 2020 se modela incentivos de precios de energía en un parque industrial, demostrando que un enfoque binivel puede simultáneamente mejorar el impacto ambiental y los beneficios económicos, utilizando un procedimiento iterativo primal-dual.

Además estudios como los de Bhavsar y Verma 2021 investigan sobre la aplicación de una política de subsidios para gestionar el riesgo de materiales peligrosos en una

red de transporte ferroviario. En este modelo, el gobierno actúa como líder, ofreciendo subsidios para incentivar al operador ferroviario (el seguidor) a usar rutas alternativas que eviten los enlaces de alto riesgo en la red, utilizando el enfoque **SLSF** (Single-Leader-Single-Follower). Los autores utilizan una reformulación de KKT para resolver el problema y aplican su método a un caso real en los Estados Unidos. Se demuestra que incluso subsidios modestos pueden resultar en una reducción significativa del riesgo.

El estudio de los problemas de dos niveles es de interés de la comunidad científica no solo porque modelan las situaciones antes mencionadas, sino porque es complejo obtener propiedades de sus soluciones así como su cálculo numérico. El concepto mismo de solución del problema binivel es complejo. La decisión del líder es solo respecto a un grupo de variables, mientras que las otras influyen en la función objetivo, pero no son decisión de él. Si para un mismo valor de las variables del líder el problema del seguidor tiene diferentes soluciones óptimas, el valor de la función objetivo del líder no estará determinado, sino que depende de cuál de los óptimos escogió el otro agente.

Para obtener las condiciones de optimalidad y los algoritmos de solución de los modelos binivel se reportan dos enfoques fundamentales. En Dempe y Zemkoho 2020a se usa la función valor extremal. Esto significa que para todo valor de x , se considera la minimización de la función objetivo del líder en el conjunto dado por las restricciones de ambos individuos y la condición de que la función objetivo del seguidor es menor o igual que el valor más pequeño que alcanza en el conjunto de soluciones factibles del seguidor.

Otro enfoque clásico consiste en sustituir el problema del nivel inferior por la condición de KKT, lo que permite transformar problemas de optimización binivel en programas matemáticos con restricciones de equilibrio (MPEC), facilitando así su resolución, ver Caselli et al. 2024; Dempe y Freiberg 2003; Dempe y Zemkoho 2020a. Conocido como *enfoque KKT* en la literatura, es una de las formas más utilizada para la resolución de problemas de dos niveles en la actualidad, ver Bouza-Allende, Aussel et al. 2021.

Dado que los problemas de optimización de ese tipo son inherentemente difíciles de resolver debido a su naturaleza **NP-hard**, ver Bard 1991; Jeroslow 1985 o incluso $\Sigma P2-hard$, ver Cerulli 2021, diciembre; Dempe y Zemkoho 2020a, se han desarrollado diversos enfoques para abordar su complejidad computacional. Sin embargo, estos enfoques suelen ser computacionalmente intensivos para problemas de gran escala, ver Cerulli 2021, diciembre. En paralelo, los algoritmos metaheurísticos, como los evolutivos, han ganado relevancia al proporcionar aproximaciones eficientes en casos no lineales o no convexos, donde las soluciones exactas son inalcanzables en tiempos razonables, ver Sinha et al. 2017. Otro enfoque destacado es el uso de métodos de descomposición, los cuales dividen el problema en subproblemas más manejables que pueden resolverse iterativamente, ver Floudas y Pardalos 1990.

Estas técnicas son particularmente útiles en aplicaciones prácticas, como los mercados de energía o los modelos de sostenibilidad, ver Siddiqui y Gabriel 2012. A pesar de estos avances, existen desafíos abiertos. La escalabilidad sigue siendo un problema crítico, ya que el crecimiento exponencial de las opciones en problemas de gran tamaño limita la aplicabilidad de los métodos exactos, ver Dempe y Zemkoho 2020a. Asimismo, los problemas no convexos carecen de garantías de convergencia hacia el óptimo global, lo que los hace especialmente difíciles de abordar. Finalmente, la incorporación de incertidumbre en los modelos agrega una capa adicional de complejidad, lo que demanda nuevos enfoques híbridos que combinen algoritmos exactos y heurísticos para mejorar la eficiencia computacional sin sacrificar la calidad de las soluciones, ver Cerulli 2021, diciembre; Sinha et al. 2017. Estos avances y desafíos reflejan la importancia de diseñar algoritmos personalizados que aprovechen las estructuras particulares de cada problema binivel. Las aplicaciones industriales, como el diseño de redes ecoindustriales y la gestión de mercados energéticos, destacan la necesidad de enfoques que equilibren precisión y tiempo de cálculo, haciendo de la optimización de dos niveles un área de investigación activa con un impacto significativo en la práctica.

Los algoritmos que se basan en las condiciones KKT incluyen una variedad de métodos, como técnicas de branch-and-bound y métodos de suavizado, ver Dempe y Zemkoho 2020a. Además, se emplean algoritmos SQP (Sequential Quadratic Programming) para resolver problemas suaves con restricciones al que se le aplica el enfoque KKT. Los problemas de optimización de dos niveles y la formulación KKT son inherentemente no convexos, lo que implica que los métodos de optimización convexa no son directamente aplicables. Esta no convexidad puede llevar a soluciones subóptimas y a dificultades para encontrar una solución global. Aunque en muchos casos las funciones involucradas en la definición del modelo sean convexas, la estructura general del problema sigue siendo no convexa.

En resumen, obtener garantías sobre soluciones en problemas de optimización de dos niveles es un desafío complejo debido a que por su no convexidad inherente, se presentan dificultades para escapar de óptimos locales, la posible falta de unicidad y la inestabilidad en las soluciones. Los algoritmos frecuentemente hallan puntos estacionarios, que no siempre corresponden a las soluciones óptimas deseadas. Por lo tanto, es crucial desarrollar métodos especializados que aborden estos problemas y permitan encontrar soluciones globales o aproximaciones adecuadas, ver Dempe y Zemkoho 2020a.

En este contexto, se ha estudiado la estructura genérica de los problemas de complementariedad (MPCC) que surgen del enfoque KKT y Fritz-John (FJ) aplicado a un problema de dos niveles. Se ha demostrado que, para una clase amplia de estos, la condición de que independencia lineal (MPCC-LICQ) se cumple en todos los puntos factibles. Sin embargo, las condiciones de complementariedad estricta (MPCC-SC) y las condiciones de segundo orden (MPCC-SOC) pueden fallar en puntos críticos

(estacionarios), incluso en situaciones genéricas, ver Bouza-Allende y Still 2012. Esta situación complica aún más la obtención de garantías sobre la solución.

Es importante señalar que existen casos singulares donde los puntos estacionarios pueden ser problemáticos, especialmente cuando el multiplicador (α) asociado a la condición de KKT del problema de nivel inferior es igual a cero. En tales circunstancias, la condición MPCC-SC puede no cumplirse, lo que podría llevar a que el método KKT no funcione adecuadamente, ver Bouza-Allende y Still 2012.

Basándose en la caracterización de los diferentes tipos de puntos estacionarios establecida por Flegel y Kanzow 2003, esta tesis propone desarrollar un generador de problemas que, dado un punto y las funciones que definen un problema de dos niveles, agregarles funciones polinomiales de primer o segundo grado de forma tal que el punto inicial dado sea un punto crítico del problema creado. Este generador facilitará el estudio del comportamiento de algoritmos conocidos en problemas con ahora al menos un punto estacionario conocido. El usuario además podrá decidir si quiere un punto crítico con multiplicadores arbitrarios o si $\alpha = \mathbf{0}$, lográndose estudiar las clases de puntos críticos que aparecen en el caso genérico, o sea en un clase amplia y significativa de los problemas generados.

La tesis está compuesta de 3 capítulos. Luego del capítulo de introducción, se mostrará la notación que se empleará, se define el problema de dos niveles con un líder y un seguidor, y se explica la teoría matemática para su transformación en un problema MPEC, así como los algoritmos de Julia que se utilizarán en ella. En el tercer capítulo se explicará la implementación algorítmica propuesta anteriormente y su correcta utilización. En el cuarto capítulo se analizarán los resultados obtenidos por el algoritmo propuesto y su comparación con algoritmos implementados en el entorno Julia. Finalmente, se presentarán las conclusiones y recomendaciones del trabajo realizado.

Capítulo 1

Preliminares

La optimización binivel es un problema de optimización donde un subconjunto de variables debe ser la solución óptima de otro problema de optimización, parametrizado por las variables restantes. Este problema tiene dos niveles jerárquicos de decisión: el nivel superior (líder) y el nivel inferior (seguidor). Este tiene dos características principales: primero, el problema del nivel inferior actúa como una restricción para el nivel superior; segundo, la solución del nivel inferior depende de las variables del nivel superior, creando una interdependencia entre ambos niveles. Por ello, el líder debe anticipar la respuesta óptima del seguidor al tomar decisiones. En términos abstractos, la optimización binivel busca minimizar una función objetivo de nivel superior, $F(x, y)$, donde x son las variables de decisión del líder y y son las variables del seguidor.

En este capítulo se abordará los conocimientos necesarios para el desarrollo de esta tesis. La primera sección trata sobre Optimización Binivel: la definición de solución basada en la existencia de una cierta cooperación entre líderes y seguidores (caso optimista) y la reformulación KKT. Luego se presentan los conceptos fundamentales de solución, regularidad y estacionariedad para Problemas Matemáticos con Restricciones de Equilibrio (MPEC). La tercera sección contiene las principales transformaciones que se aplican a las restricciones de complementariedad para la solución numérica del modelo. Por último, se presenta la modelación de los problemas binivel en el lenguaje de programación Julia y los métodos seleccionados implementados en este lenguaje para su resolución.

Optimización Binivel

El problema de optimización binivel en general se define de la siguiente manera

$$\begin{aligned} & \min_x F(x, y) \\ & \text{s.a.} \begin{cases} x \in \mathcal{T} \\ y \in S(x) = \arg \min_y \{f(x, y) \mid y \in \mathcal{H}\} \\ (x, y) \in \mathcal{M}^0 \end{cases} \end{aligned} \quad (1.1)$$

En la mayoría de las aplicaciones, los problemas de optimización del nivel inferior y superior son modelos de programación matemática, por lo que, sin pérdida de generalidad, se asume que $\mathcal{T} = \mathbb{R}^n$, $\mathcal{H}$ es $\{y \in \mathbb{R}^m \mid v_j(x, y) \leq 0, j = 1, \dots, q\}$ y $\mathcal{M}^0 = \{(x, y) \in \mathbb{R}^{n+m}, g_i(x, y) \leq 0, i = 1, \dots, q\}$. El problema es

$$\begin{aligned} & \min_x F(x, y) \\ & \text{sujeto a} \\ & g_i(x, y) \leq 0, i = 1, \dots, q, \\ & y \in \arg \min_y \{f(x, y) \mid v_j(x, y) \leq 0, j = 1, \dots, q\} \end{aligned} \quad (1.2)$$

donde $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$, $F(x, y) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $g_i(x, y) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $i = 1, \dots, q$, $f(x, y) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $v_j(x, y) : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $j = 1, \dots, s$. En el enfoque optimista, se asume que el seguidor, que actúa en el nivel inferior, elegirá la solución más favorable para el líder, quien toma decisiones en el nivel superior. Este es considerado más tratable y, en ciertas situaciones favorables, puede simplificarse a un problema convexo. Además, en el contexto de múltiples objetivos, el enfoque optimista permite alcanzar el mejor frente de Pareto posible, ver Dempe y Zemkoho 2020a.

$$\begin{aligned} & \min_x \inf_{y \in S(x)} F(x, y) \\ & g_i(x, y) \leq 0, i = 1, \dots, q, \\ & \text{con } S(x) = \arg \min_y \{f(x, y) \mid v(x, y) \leq 0\} \end{aligned} \quad (1.3)$$

Figura 1.1: Problema de optimización bajo el enfoque optimista.

Por otro lado, el enfoque pesimista asume que el seguidor seleccionará la opción menos favorable para el líder entre las soluciones óptimas disponibles, el cual es más complejo de resolver y puede incluso no tener solución. A menudo, se requieren reformulaciones para abordar estos problemas, lo que lo convierte en un reto teórico y computacional significativo en situaciones de múltiples objetivos, conduce al peor frente de Pareto

posible, ver Sinha et al. 2017.

$$\begin{aligned} & \min_x \sup_{y \in S(x)} F(x, y) \\ & g_i(x, y) \leq 0, \quad i = 1, \dots, q, \\ & \text{con } S(x) = \operatorname{argmin}_y \{f(x, y) \mid v(x, y) \leq 0\} \end{aligned} \quad (1.4)$$

Figura 1.2: Problema de optimización bajo el enfoque pesimista.

Es relevante destacar que la mayoría de la literatura se centra en el enfoque optimista debido a su mayor facilidad de tratamiento. Sin embargo, el otro también tiene su utilidad, especialmente en la modelación de situaciones donde se considera la aversión al riesgo, ver Dempe y Zemkoho 2020a. En este contexto, los términos "líder" y "seguidor" se utilizan para describir los roles en el modelo a optimizar; el líder toma decisiones considerando las posibles reacciones del seguidor, quien a su vez reacciona seleccionando su mejor opción, ver Sinha et al. 2017.

Dado que en la tesis trataremos sobre problemas binivel de enfoque optimista mostraremos algunos resultados referidos al modelo resultante.

Primeramente es importante notar el siguiente resultado, ver Schmidt y Beck 2021.

Proposition 1 *El problema de optimización binivel optimista (1.3) se formula equivalentemente como:*

$$\begin{aligned} & \min_{x, y} F(x, y) \\ & \text{s.t.} \quad g_i(x, y) \leq 0, \quad i = 1 \dots q, \\ & \quad \quad y \in S(x), \end{aligned}$$

donde $S(x)$ es el conjunto de soluciones óptimas del problema parametrizado por x

$$\min_{y \in Y(x)} f(x, y) \text{ s.t. } v_j(x, y) \leq 0, \quad j = 1 \dots s,$$

De ahí que a partir de ahora se considerará el problema (1.5) y se le llamará indistintamente problema de dos niveles. Los problemas de dos niveles pueden ser reformulados en un problema de un solo nivel al reemplazar el problema del nivel inferior por las condiciones KKT de este en las restricciones del primer nivel.

$$\begin{aligned} & \min_{x, y, \lambda_j} F(x, y) \\ & \text{s.a.} \quad \begin{cases} g_i(x, y) \leq 0, \quad i = 1 \dots q, \\ \nabla_y f(x, y) + \sum_{j=1}^s \nabla_y v_j(x, y) \lambda_j = 0, \\ v_j(x, y) \leq 0, \quad j = 1 \dots s, \\ v_j(x, y) \lambda_j = 0, \quad j = 1 \dots s, \\ \lambda_j \geq 0, \quad j = 1 \dots s. \end{cases} \end{aligned} \quad (1.5)$$

Los tres últimos grupos de restricciones expresan que v y λ están restringidas en signo y que al menos una es 0. Estas condiciones son conocidas como **restricciones de complementariedad**. Estos modelos corresponden a la clase de problemas de programación matemática con restricciones de complementariedad (MPEC). Como se mencionó en la introducción, existe un enfoque alternativo mediante un problema de programación matemática utilizando la función del valor extremal. Sin embargo, esta función carece de una expresión explícita, exceptuando casos muy específicos. La ventaja del método KKT para MPEC radica en que proporciona una forma explícita, aunque requiere ciertas condiciones de regularidad del conjunto, como la Condición de Calificación de Restricciones de Independencia Lineal (LICQ), por sus siglas en inglés, de las restricciones activas. En caso de no cumplirse estas condiciones, podrían existir puntos óptimos que no satisfagan las condiciones KKT. A continuación, presentamos los resultados de esta área necesarios para el desarrollo de esta tesis.

Programación Matemática con Restricciones de Equilibrio (MPEC)

Un **problema de Programación Matemática con Restricciones de Equilibrio (MPEC)** es un tipo de problema de optimización no lineal que incluye restricciones de equilibrio, específicamente, restricciones de complementariedad.

$$\begin{aligned}
 & \text{mín} && f(z) \\
 & \text{s.t.} && g_i(z) \leq 0, \quad i = 1, \dots, q, \\
 & && h_k(z) = 0, \quad k = 1, \dots, m, \\
 & && G_j(z) \geq 0, \quad j = 1, \dots, s, \quad H_j(z) \geq 0, \quad j = 1, \dots, s, \\
 & && G_j(z)^T H_j(z) = 0, \quad j = 1, \dots, s.
 \end{aligned} \tag{1.6}$$

Definición de MPEC

donde $f : \mathbb{R}^{\hat{n}} \rightarrow \mathbb{R}$, $g : \mathbb{R}^{\hat{n}} \rightarrow \mathbb{R}$, $h : \mathbb{R}^{\hat{n}} \rightarrow \mathbb{R}$, $G : \mathbb{R}^{\hat{n}} \rightarrow \mathbb{R}$, y $H : \mathbb{R}^{\hat{n}} \rightarrow \mathbb{R}$ son funciones continuamente diferenciables. El término de complementariedad se debe a la existencia de las restricciones homónimas

$$G_j(z) \geq 0, H_j(z) \geq 0, \quad G_j(z)^T H_j(z) = 0, \quad j = 1, \dots, s. \tag{1.7}$$

Debido a las restricciones de complementariedad, los MPEC no cumplen con las condiciones de regularidad estándar, lo que hace que las condiciones de KKT no sean directamente aplicables como condiciones de optimalidad de primer orden. Los MPEC son utilizados para modelar problemas donde existen restricciones de equilibrio, como problemas de ingeniería y economía, ver Dempe y Zemkoho 2020a; Flegel y Kanzow 2003.

A continuación como en Flegel y Kanzow 2003, se introduce las siguientes definiciones, útiles para la obtención de condiciones necesarias de optimalidad.

Definición 1.1 (Conjunto de Índices) *Dado un vector factible z^* del MPEC (1.6), definimos los siguientes conjuntos de índices:*

$$\begin{aligned} J_G &:= J_G(z^*) := \{i | G_i(z^*) = 0, H_i(z^*) > 0\} \\ J_{GH} &:= J_{GH}(z^*) := \{i | G_i(z^*) = 0, H_i(z^*) = 0\} \\ J_H &:= J_H(z^*) := \{i | G_i(z^*) > 0, H_i(z^*) = 0\} \end{aligned} \quad (1.8)$$

Para definir condiciones de regularidad para MPEC, introducimos el siguiente problema, dependiente de z^* :

Definición 1.2 (Programa No Lineal Relacionado (RNLP)) *Un Programa No Lineal Relacionado $RNLP_{a,b} := RNLP_{a,b}(z^*)$ es:*

$$\begin{aligned} \min \quad & f(z) \\ \text{s.t.} \quad & g_i(z) \leq 0, \quad i = 1, \dots, q, \\ & h_k(z) = 0, \quad k = 1, \dots, m, \\ & G_j(z) = 0, \quad j \in J_G \cup J_a, \quad G_j(z) \geq 0, \quad j \in J_H, \\ & H_j(z) = 0, \quad j \in J_H \cup J_b, \quad H_j(z) \geq 0, \quad j \in J_G, \end{aligned} \quad (1.9)$$

Problema No Lineal Relacionado (RNLP)

donde $a \cup b = J_{GH}$.

El RNLP (1.9) puede usarse para definir variantes de regularidad y condiciones necesarias de optimalidad adecuadas para MPEC como son la restricción estándar de independencia lineal (LICQ en su forma abreviada) y la de punto de KKT.

Definición 1.3 (MPEC-LICQ) *El MPEC (1.6) se dice que satisface la MPEC-LICQ en un vector factible z^* si los correspondientes $RNLP_{a,b}(z^*)$ satisfacen la LICQ en ese vector z^* .*

Cabe destacar que si la LICQ se cumple en $RNLP_{a,b}(z^*)$ para una partición (a, b) se cumple para todas.

En el contexto de los MPEC en Flegel y Kanzow 2003 se exponen varios tipos de puntos estacionarios que son cruciales para analizar la optimalidad, los cuales son los siguientes:

Definición 1.4 (Punto Factible) *Un punto factible z^* del MPEC se llama débilmente estacionario si existe un multiplicador de Lagrange $(\mu, \alpha, \beta, \gamma)$ tal que se cumplen las siguientes condiciones:*

$$\begin{aligned}
& \nabla f(z^*) + \sum_{i=1}^q \mu_i \nabla g_i(z^*) + \sum_{k=1}^{q_0} \alpha_k \nabla h_k(z^*) - \sum_{j=1}^s [\beta_j \nabla G_j(z^*) + \gamma_j \nabla H_j(z^*)] = \mathbf{0} \\
& \beta_j, j \in J_G \cup J_{GH} \text{ libre, } \beta_j, j \in J_H = 0, \\
& \gamma_j, j \in J_H \cup J_{GH} \text{ libre, } \gamma_j, j \in J_G = 0, \\
& g_i(z^*) \leq 0, \quad \mu_i \geq 0, \quad \mu_i g_i(z^*) = 0, i = 1, \dots, q.
\end{aligned} \tag{1.10}$$

Este concepto de estacionariedad es equivalente al cumplimiento de la estacionariedad clásica en el problema $RNLP_{J_{GH}, J_{GH}}$. De ahí que es una condición relativamente débil. Existen conceptos más fuertes de estacionariedad que se derivan y estudian en otros lugares. En particular, se tienen las siguientes definiciones:

Definición 1.5 (Punto C-estacionario) *El punto z^* es: C-estacionario si, para cada $j \in J_{GH}$, $\beta_j \gamma_j \geq 0$ se cumple.*

Definición 1.6 (Punto M-estacionario) *El punto z^* es: M-estacionario si, para cada $j \in J_{GH}$, o bien $\beta_j, \gamma_j > 0 \vee \beta_j \gamma_j = 0$.*

Definición 1.7 (Punto Fuertemente estacionario) *El punto z^* es: fuertemente estacionario si, para cada $j \in J_{GH}$, $\beta_j, \gamma_j \geq 0$.*

La C y la M estacionariedad son condiciones necesarias de optimalidad bajo ciertas condiciones de regularidad, más débiles que la MPEC-LICQ. La fuerte conlleva al siguiente resultado:

Teorema 1.1 *Sea $z^* \in \mathbb{R}^n$ un mínimo local del MPEC (1.6). Si MPEC-LICQ se cumple en z^* , entonces existe un único multiplicador de Lagrange tal que (z^*, μ^*) es fuertemente estacionario.*

O sea el punto z^* es un punto estacionario del problema relajado

$$\begin{aligned}
& \text{mín} && f(z) \\
& \text{s.t.} && g_i(z) \leq 0, i = 1, \dots, q, \\
& && h_k(z) = 0, k = 1, \dots, m, \\
& && G_j(z) = 0, j \in J_G \quad G_j(z) \geq 0, j \in J_{GH}, \quad G_j(z) \geq 0, j \in J_H, \\
& && H_j(z) = 0, j \in J_H \quad H_j(z) \geq 0, j \in J_{GH}, \quad H_j(z) \geq 0, j \in J_G, \\
& && \text{Problema No Lineal Relajado}
\end{aligned} \tag{1.11}$$

Los algoritmos buscan al menos converger a puntos de este tipo. En la próxima sección se revisarán las reformulaciones que se resuelven las bibliotecas de Julia.

Métodos de Reformulación para Optimización Binivel

La optimización binivel presenta desafíos particulares debido a su naturaleza jerárquica y las condiciones de complementariedad resultantes. A continuación, se presentan los principales métodos de reformulación implementados en la literatura, basados en la transformación del MPEC (1.5).

1.3.1. Método Big-M

El método Big-M (Fortuny-Amat y McCarl) es una técnica fundamental para reformular problemas de optimización binivel en problemas MPEC. Este método aborda específicamente las condiciones de complementariedad que surgen en estas reformulaciones, transformando el problema original en un problema de programación lineal mixta entera (MILP).

La reformulación mediante Big-M introduce un parámetro M suficientemente grande y variables binarias para transformar las condiciones de complementariedad no lineales en restricciones lineales. Para cada condición de complementariedad $v_j(x, y)\lambda_j = 0$ en (1.5), el método introduce una variable binaria $\delta_j \in \{0, 1\}$ y cotas superiores M_p, M_d bajo las siguientes restricciones:

$$\begin{aligned} v_j(x, y) &\geq -M_p(1 - \delta_j) \\ \lambda_j &\leq M_d\delta_j \\ \delta_j &\in \{0, 1\} \end{aligned} \tag{1.12}$$

donde M_p y M_d son valores grandes para las variables primales y duales, respectivamente. La efectividad del método depende crucialmente de la selección apropiada de estos valores, que deben ser suficientemente grandes para no excluir la solución óptima, pero no excesivamente grandes para evitar inestabilidades numéricas Garcia et al. 2022.

1.3.2. Método SOS1

El método de Conjuntos Ordenados Especiales tipo 1 (SOS1) evita el uso de parámetros Big-M mediante restricciones de tipo conjunto. Para cada par complementario (v_j, λ_j) , $j = 1, \dots, s$:

$$\begin{aligned} v_j(x, y) &= s_j \\ [s_j; \lambda_j] &\in \text{SOS1} \end{aligned} \tag{1.13}$$

donde $\text{SOS1} = \{(a, b) \in \mathbb{R} : a, b \geq 0, ab = 0\}$

Esta restricción fuerza que al menos una variable en el par sea cero, preservando la no linealidad original sin necesidad de cotas. Note que SOS1 es un cono. Esta es una reformulación del MPEC como un problema de optimización cónica. Es particularmente eficaz en MPECs con restricciones del tipo $u \in K$, donde K es un cono, pues sigue siendo un problema de optimización cónica y se pueden aplicar algoritmos específicos para la resolución de este tipo de modelos, Garcia et al. 2022. La dificultad principal es que el cono es no convexo. Una aplicación de este enfoque se encuentra en Siddiqui y Gabriel 2012.

1.3.3. Método ProductMode

El método ProductMode representa un enfoque directo para manejar las condiciones de complementariedad en su forma de producto original. Este método es particularmente útil cuando se trabaja con solucionadores de programación no lineal (NLP), aunque no garantiza la optimalidad global.

La implementación del ProductMode mantiene la restricción de complementariedad en su forma original:

$$v_j(x, y) \cdot \lambda_j \leq t \quad (1.14)$$

donde $t > 0$ es un parámetro de regularización pequeño. Esta formulación, aunque no satisface las condiciones de calificación de restricciones estándar, es útil para obtener soluciones iniciales y puede ser especialmente efectiva cuando se combina con solucionadores NLP, ver Garcia et al. 2022. sin embargo, bajo condiciones naturales, la sucesión que genera converge a puntos C estacionarios y bajo regularidad hasta fuertemente estacionarios, ver Scholtes 2001.

La Tabla 1.1 resume los requisitos y características de cada método:

Tabla 1.1: Comparación de métodos de reformulación

Método	Solver Requerido	Ventajas
Big-M	MIP	Estabilidad numérica controlada
SOS1	MIP con SOS1	Sin parámetros ad-hoc
ProductMode	NLP	Manejo de no linealidades

Para casos con restricciones no lineales $v_j(x, y)$, se recomienda combinar ProductMode con técnicas de linealización por tramos Garcia et al. 2022, Apéndice B. Todos estos métodos están implementados en BilevelJuMP.jl, permitiendo experimentación con reformulaciones que se pueden encontrar en Garcia et al. 2022, Sección 4.

Modelación en Julia

El lenguaje de programación Julia se ha posicionado como una herramienta revolucionaria en el ámbito de la computación científica y la optimización matemática. Desarrollado en el MIT, Julia fue diseñado desde sus cimientos para abordar el denominado "problema de los dos lenguajes", donde tradicionalmente los investigadores se veían obligados a prototipar en un lenguaje de alto nivel como Python y luego reimplementar en C++ o Fortran para obtener rendimiento óptimo. Julia logra combinar la facilidad de uso de lenguajes dinámicos como Python con el rendimiento de lenguajes compilados como C, gracias a su sistema de tipos y su compilador LLVM. Una de las características más destacadas de Julia es su sintaxis expresiva y natural para la formulación matemática. El lenguaje permite escribir código que se asemeja notablemente a las expresiones matemáticas tradicionales, facilitando la traducción directa de modelos matemáticos a código ejecutable. Esta característica es particularmente valiosa en el campo de la optimización, donde la claridad en la expresión de modelos matemáticos es crucial. Julia alcanza velocidades de ejecución comparables a C y Fortran, superando significativamente a Python en cálculos numéricos intensivos, con mejoras de rendimiento que pueden alcanzar órdenes de magnitud en determinadas aplicaciones. La interoperabilidad de Julia con otros lenguajes de programación constituye una ventaja significativa para proyectos de optimización complejos. A través de paquetes como PyCall, RCall, y JavaCall, Julia puede integrarse perfectamente con bibliotecas establecidas en Python, R y Java. Esta capacidad permite aprovechar el extenso ecosistema de estos lenguajes mientras se mantiene el rendimiento superior de Julia en los cálculos críticos de optimización. Además, Julia puede llamar directamente a funciones de C y Fortran sin overhead adicional, permitiendo la reutilización de código legacy optimizado y la integración con sistemas empresariales existentes. El ecosistema de optimización en Julia está encabezado por JuMP (Julia for Mathematical Programming), un lenguaje de modelado algebraico embebido que permite expresar problemas de optimización de manera natural y eficiente. JuMP se destaca por su capacidad para manejar problemas de gran escala y su integración perfecta con diversos solucionadores, tanto comerciales como de código abierto. La extensión BilevelJuMP amplía estas capacidades al dominio de la optimización binivel, permitiendo la formulación y resolución de problemas jerárquicos complejos, un área de creciente importancia en la investigación de optimización moderna.

Julia facilita la integración con solucionadores de alto rendimiento como Ipopt para optimización no lineal, SCIP para programación entera mixta, y HiGHS para programación lineal. La interfaz nativa de Julia con estos solucionadores minimiza el overhead de comunicación, resultando en tiempos de resolución significativamente menores comparados con interfaces basadas en Python. Por ejemplo, en problemas de optimización no lineal de gran escala, la combinación de JuMP con Ipopt puede

ejecutarse hasta 10 veces más rápido que implementaciones equivalentes en Python utilizando Pyomo o AMPL, lo que demuestra la eficiencia del ecosistema de Julia en aplicaciones prácticas.

La capacidad de Julia para escalar eficientemente en entornos de computación distribuida y GPUs es particularmente relevante para problemas de optimización de gran escala. El lenguaje incluye soporte nativo para computación paralela y distribuida, permitiendo la paralelización de algoritmos de optimización con mínimas modificaciones al código. Paquetes como DistributedArrays y CUDA.jl facilitan la implementación de algoritmos de optimización en clusters y GPUs, respectivamente, manteniendo la sintaxis clara y expresiva característica de Julia.

En el ámbito específico de la optimización matemática, Julia presenta ventajas significativas sobre Python, particularmente en términos de rendimiento y expresividad. Mientras que Python requiere de bibliotecas como Numpy y Scipy para operaciones numéricas eficientes, Julia proporciona estas capacidades de manera nativa. En problemas de optimización complejos, las implementaciones en Julia con JuMP típicamente requieren menos código y se ejecutan más rápidamente que sus equivalentes en Python utilizando Pyomo o CVXPY. Estudios comparativos han demostrado que Julia con JuMP puede resolver problemas de optimización típicos entre 2 y 10 veces más rápido que implementaciones equivalentes en Python, dependiendo de la naturaleza y escala del problema. Esta ventaja se hace más pronunciada en problemas que requieren múltiples evaluaciones de la función objetivo o restricciones complejas, atribuyéndose principalmente a la compilación Just-In-Time de Julia, la ausencia de overhead en la llamada a funciones, y la integración más eficiente con los solucionadores de optimización.

Por los argumentos anteriores en este trabajo utilizaremos este lenguaje de programación con principalmente los módulos **JuMP** y **BilevelJuMP**, de los cuales se describe a continuación.

JuMP

JuMP (Julia for Mathematical Programming) es una biblioteca de modelado algebraico integrada en Julia que permite formular y resolver problemas de optimización matemática de manera eficiente y expresiva. Esta biblioteca está diseñada para abordar problemas de programación matemática que pueden expresarse en la forma general:

$$\min_z F(z) \tag{1.15}$$

sa:

$$\begin{aligned} g_i(z) &\leq 0, \quad i = 1, \dots, q, \\ h_k(z) &= 0, \quad k = 1, \dots, q_0, \end{aligned} \tag{1.16}$$

donde $z \in \mathbb{R}^p \times \mathbb{Z}^{\hat{n}-p}$ representa el vector de variables de decisión, que puede incluir tanto componentes continuos como discretos. La función $F(z)$ representa el objetivo a minimizar, mientras que $g_i(z)$ y $h_k(z)$ representan las restricciones de desigualdad e igualdad, respectivamente.

JuMP proporciona una interfaz intuitiva y matemáticamente rigurosa para la construcción de estos modelos mediante un conjunto de macros especializadas. La macro **@variable** se utiliza para declarar las variables de decisión, permitiendo especificar su naturaleza (continua o discreta) y sus dominios. Por ejemplo, podemos declarar variables continuas no negativas o variables enteras acotadas. La función objetivo se define mediante la macro **@objective**, que acepta expresiones lineales, cuadráticas o no lineales. JuMP distingue automáticamente entre estos tipos de expresiones y selecciona los métodos de solución apropiados. La sintaxis es clara y cercana a la notación matemática tradicional, lo que facilita la traducción de modelos matemáticos a código ejecutable. Para la especificación de restricciones, JuMP ofrece dos macros principales: **@constraint** para restricciones lineales y cuadráticas, y **@NLconstraint** para restricciones no lineales generales. La biblioteca identifica automáticamente la naturaleza de las restricciones y las procesa de manera eficiente. Las restricciones pueden incluir desigualdades, igualdades y expresiones más complejas que involucren funciones no lineales. Una característica particularmente potente de JuMP es su integración con el paquete **Complementarity.jl**, ver documentación Chkwon 2025, que permite la formulación de problemas de complementariedad matemática (MPECs). Esto es especialmente útil para problemas de optimización jerárquica o de equilibrio, donde las condiciones de complementariedad son fundamentales. Las restricciones de complementariedad se pueden expresar de manera natural utilizando la sintaxis proporcionada por estas bibliotecas.

Al resolver un problema de optimización, JuMP proporciona información detallada sobre el estado de la solución mediante un conjunto de estados predefinidos. Estos estados, conocidos como estados primales, permiten interpretar el resultado obtenido. El estado **NO_SOLUTION** indica la ausencia de un vector resultado, mientras que **FEASIBLE_POINT** confirma la obtención de una solución que satisfaga todas las restricciones del problema. En casos donde la precisión numérica juega un papel importante, **NEARLY_FEASIBLE_POINT** señala soluciones que son factibles bajo una relajación de las tolerancias en las restricciones. Para problemas que no tienen solución, JuMP puede proporcionar diferentes certificados. El estado **INFEASIBLE_POINT** indica que el punto encontrado viola las restricciones, mientras que **INFEASIBILITY_CERTIFICATE** proporciona una prueba matemática de que el problema es no factible. En situaciones numéricamente desafiantes, **NEARLY_INFEASIBILITY_CERTIFICATE** representa un certificado de no factibilidad bajo criterios relajados. La biblioteca también puede identificar problemas mal planteados mediante los estados **REDUCTION_CERTIFICATE**

y **NEARLY_REDUCTION_CERTIFICATE**, que indican la existencia de certificados de mal planteamiento exactos o aproximados, respectivamente. Para casos especiales o no contemplados en las categorías anteriores, se utilizan los estados **UNKNOWN_RESULT_STATUS** y **OTHER_RESULT_STATUS**.

Además, JuMP incorpora la enumeración **TerminationStatusCode**, que explica la razón por la cual el optimizador dejó de ejecutarse en la última llamada a **optimize!**. Los posibles valores incluyen: **OPTIMIZE_NOT_CALLED**, cuando el algoritmo no ha comenzado; **OPTIMAL**, si se encuentra una solución óptima global; **INFEASIBLE**, cuando el optimizador concluye que no existen soluciones factibles; **DUAL_INFEASIBLE**, si no existe una solución dual acotada y, además, se conoce una solución primal, lo que sugiere que el problema es no acotado; **LOCALLY_SOLVED**, cuando el algoritmo converge a un punto estacionario o una solución óptima local, pero sin garantías globales; **LOCALLY_INFEASIBLE**, si se alcanza un punto no factible sin garantía de que no existan soluciones factibles; **INFEASIBLE_OR_UNBOUNDED**, cuando el optimizador determina que el problema es no factible o no acotado, lo que puede ocurrir en la fase de preprocesamiento; **ALMOST_OPTIMAL**, cuando se encuentra una solución óptima global bajo tolerancias relajadas; **ALMOST_INFEASIBLE**, si se concluye que no existe una solución factible bajo tolerancias relajadas; **ALMOST_DUAL_INFEASIBLE**, cuando no se encuentra una cota dual válida bajo tolerancias relajadas; **ALMOST_LOCALLY_SOLVED**, si el algoritmo converge a un punto estacionario o solución local dentro de tolerancias relajadas; **ITERATION_LIMIT**, si el algoritmo se detiene tras alcanzar el número máximo de iteraciones permitidas; **TIME_LIMIT**, cuando el tiempo de ejecución excede el límite definido por el usuario; **NODE_LIMIT**, si un algoritmo basado en branch-and-bound explora un número máximo de nodos; **SOLUTION_LIMIT**, cuando se alcanza el número requerido de soluciones; **MEMORY_LIMIT**, si el optimizador se detiene debido a la falta de memoria; **OBJECTIVE_LIMIT**, cuando el algoritmo encuentra una solución mejor que el límite mínimo establecido por el usuario; **NORM_LIMIT**, si la norma de un iterado se vuelve demasiado grande; **OTHER_LIMIT**, cuando el algoritmo se detiene debido a un límite que no pertenece a las categorías anteriores; **SLOW_PROGRESS**, cuando el algoritmo no puede continuar avanzando de manera efectiva hacia la solución; **NUMERICAL_ERROR**, si ocurre un error numérico irrecuperable; **INVALID_MODEL**, cuando el modelo especificado es inválido; **INVALID_OPTION**, si se proporciona una opción de entrada inválida; **INTERRUPTED**, cuando el algoritmo es interrumpido manualmente; y **OTHER_ERROR**, si ocurre un error no contemplado en los estados anteriores.

Una vez que el modelo está completamente especificado, la función **optimize!** se utiliza para resolver el problema utilizando el solucionador seleccionado. JuMP es compatible con una amplia gama de solucionadores, tanto de código abierto como

comerciales, y selecciona automáticamente el más apropiado según la estructura del problema (lineal, cuadrático, no lineal, entero mixto, etc.). Es importante destacar que JuMP maneja eficientemente la diferenciación automática para problemas no lineales, lo que permite calcular gradientes y Hessianas de manera precisa y eficiente, mejorando significativamente el rendimiento y la robustez de los métodos de optimización utilizados.

Para más detalles sobre la biblioteca, ver Lubin et al. 2023.

BilevelJuMP

BilevelJuMP.jl es una extensión especializada de Julia diseñada específicamente para abordar problemas de optimización binivel, también conocidos como problemas de optimización jerárquica o de dos niveles. Esta biblioteca se construye sobre la base de JuMP, aprovechando su sintaxis intuitiva y capacidades de modelado mientras agrega funcionalidades específicas para la formulación y resolución de problemas jerárquicos. Además los estados primales y estados de terminación son análogos a los proporcionados por JuMP, manteniendo así la consistencia en la interfaz de programación. Esto significa que los usuarios familiarizados con JuMP encontrarán una experiencia similar al consultar el estado de las soluciones y la terminación de los problemas de optimización binivel.

Este facilita la modelación de esta jerarquía propia de los problemas binivel permitiendo que el problema del seguidor se formule utilizando la sintaxis familiar de JuMP. Esto significa que los usuarios pueden especificar restricciones lineales y cuadráticas, definir variables tanto continuas como enteras, y trabajar con diferentes tipos de funciones objetivo, manteniendo la claridad y expresividad características de JuMP. De manera análoga, el problema del nivel superior se gestiona con la misma flexibilidad, permitiendo la especificación de sus propias restricciones y función objetivo.

Una característica notable de BilevelJuMP.jl es su capacidad para manejar diferentes reformulaciones de las restricciones de complementariedad en problemas MPEC (Mathematical Programs with Equilibrium Constraints). Los usuarios pueden experimentar con diversas técnicas de reformulación, incluyendo el método SOS1 (Special Ordered Sets of type 1), la reformulación de McCarl (conocida como Big-M) y ProductMode, descritos anteriormente, adaptando la resolución del problema según las necesidades específicas del caso.

La versatilidad de BilevelJuMP.jl se evidencia en su compatibilidad con diversos tipos de solucionadores. La biblioteca puede utilizar tanto solucionadores de programación lineal mixta entera (MIP) como solucionadores de programación no lineal (NLP), seleccionando el más apropiado según las características particulares del problema y la reformulación elegida. Esta flexibilidad permite abordar una amplia gama de problemas binivel con diferentes estructuras y complejidades.

Sin embargo, es importante reconocer las limitaciones inherentes a BilevelJuMP.jl. Este no permite problemas no convexos, además puede encontrar dificultades al manejar problemas altamente no lineales o con estructuras de optimización particularmente complejas que desafían la representación estándar en la sintaxis de JuMP. Algunas restricciones específicas podrían requerir transformaciones adicionales que incrementan la complejidad del modelo, y en casos de problemas de gran escala, el rendimiento del solucionador puede convertirse en un factor crítico que afecte la eficiencia de la resolución.

Adicionalmente, la formulación y resolución de problemas muy específicos o especializados podría no estar completamente optimizada dentro del paquete, lo que podría requerir adaptaciones o consideraciones especiales por parte del usuario. Estas limitaciones, aunque importantes de considerar, no disminuyen la utilidad general de BilevelJuMP.jl como una herramienta poderosa para la optimización binivel, sino que más bien definen el alcance de su aplicabilidad óptima.

Para más detalles sobre la biblioteca, ver Garcia et al. 2022.

Teniendo en cuenta la posibilidad de que los métodos de solución converjan a puntos estacionarios de distinto tipo, resulta interesante comprobar si, conocido un punto de esta naturaleza, el algoritmo logra obtener un punto con mejor evaluación de la función objetivo. Esta es la motivación para el desarrollo del generador de problemas prueba, objeto de esta tesis y que se presentará en el próximo capítulo.

Symbolics

Symbolics.jl representa un avance significativo en el campo de la computación simbólica dentro del ecosistema de Julia, ofreciendo un conjunto robusto de herramientas para la manipulación algebraica y el cálculo simbólico. Esta biblioteca se destaca por su capacidad para manejar expresiones matemáticas de manera abstracta, permitiendo operaciones como diferenciación, integración y simplificación simbólica con una eficiencia notable. El diseño de Symbolics.jl aprovecha las características fundamentales de Julia, como su sistema de tipos y compilación JIT, para proporcionar un rendimiento excepcional en comparación con sistemas tradicionales de computación simbólica. La biblioteca permite la creación y manipulación de variables simbólicas que pueden representar cualquier expresión matemática, implementando capacidades robustas de diferenciación automática que incluyen el cálculo de derivadas parciales y totales. Una característica particularmente relevante es su sistema de simplificación de expresiones, que utiliza algoritmos avanzados para reducir automáticamente la complejidad de expresiones matemáticas extensas. La integración de Symbolics.jl con el resto del ecosistema Julia es seamless, lo que facilita su uso en conjunto con otros paquetes orientados al cálculo numérico y la resolución de ecuaciones diferenciales. Esta interoperatividad resulta especialmente valiosa en aplicaciones que requieren

tanto manipulación simbólica como evaluación numérica, como el modelado matemático, el análisis numérico y la optimización. La biblioteca sobresale en la generación automática de expresiones optimizadas para cálculos numéricos, lo que la hace particularmente útil en el contexto de la optimización matemática, donde la eficiencia en la evaluación de funciones objetivo y restricciones es crucial. Además, su capacidad para derivar expresiones simbólicas de gradientes y hessianas de manera automática la convierte en una herramienta fundamental para problemas de optimización que requieren información de derivadas de orden superior.

Para más detalles sobre la biblioteca, ver Gowda et al. 2021

LinearAlgebra

LinearAlgebra constituye uno de los módulos fundamentales del núcleo de Julia, proporcionando una implementación robusta y eficiente de operaciones de álgebra lineal esenciales para la computación científica y la optimización matemática. Este módulo se distingue por su capacidad para manejar operaciones matriciales complejas con un rendimiento comparable al de bibliotecas altamente optimizadas como BLAS y LAPACK, mientras mantiene una sintaxis intuitiva y cercana a la notación matemática tradicional. LinearAlgebra implementa una amplia gama de operaciones matriciales fundamentales, incluyendo descomposiciones matriciales (LU, QR, SVD, Cholesky), cálculo de autovalores y autovectores, y operaciones básicas como productos matriciales y vectoriales, con optimizaciones específicas para diferentes tipos de matrices (densas, dispersas, simétricas, triangulares). La biblioteca destaca por su sistema de tipos especializados que permite representar eficientemente estructuras matriciales específicas, como matrices simétricas, hermíticas o triangulares, aprovechando sus propiedades matemáticas para optimizar tanto el almacenamiento como las operaciones computacionales. Una característica particularmente relevante es su integración natural con el sistema de despacho múltiple de Julia, que permite que las operaciones se optimicen automáticamente según los tipos de datos involucrados, resultando en un rendimiento excepcional sin sacrificar la claridad del código. Esta eficiencia es crucial en el contexto de la optimización matemática, donde las operaciones de álgebra lineal son frecuentemente el cuello de botella computacional en la evaluación de funciones objetivo y restricciones. Además, el módulo proporciona funcionalidades avanzadas como la factorización de matrices dispersas y el cálculo de normas matriciales, que son esenciales en algoritmos de optimización numérica y métodos iterativos de resolución de sistemas lineales.

Para más detalles sobre la biblioteca, ver Language 2025

Capítulo 2

Detalles de Implementación y Experimentos

En este capítulo se mostrará la forma y los pasos para generar el problema deseado en el contexto de un generador de puntos estacionarios para problemas binivel. Este capítulo aborda el diseño y la implementación del generador, explicando detalladamente las metodologías empleadas, los criterios necesarios para asegurar la validez de los puntos obtenidos y los algoritmos utilizados en cada etapa del proceso.

Notaciones:

El problema binivel optimista, definido en (1.2), consiste en

$$\begin{aligned} & \min_x F(x, y) \\ & \text{sujeto a} \\ & g_i(x, y) \leq 0, \quad i = 1, \dots, q, \\ & y \in \operatorname{argmin}_y \{f(x, y) \mid v_j(x, y) \leq 0, \quad j = 1, \dots, s, \} \end{aligned} \tag{2.1}$$

Luego el modelo se puede definir usando solo las funciones

- $F(x, y)$: Función del líder.
- $g_i(x, y)$: Restricciones de desigualdad del líder, $i = 1, \dots, q$.
- $f(x, y)$: Función del seguidor.

- $v_j(x, y)$: Restricciones de desigualdad del seguidor, $j = 1 \dots s$.

A este modelo se le aplicará el enfoque KKT, obteniéndose

$$\begin{aligned} & \min_{x, y, \lambda_j} F(x, y) \\ & s.a \begin{cases} g_i(x, y) \leq 0, i = 1 \dots q, \\ \nabla_y f(x, y) + \sum_{j=1}^s \nabla_y v_j(x, y) \lambda_j = 0, \\ v_j(x, y) \leq 0, j = 1 \dots s, \\ v_j(x, y) \lambda_j = 0, j = 1 \dots s, \\ \lambda_j \geq 0, j = 1 \dots s. \end{cases} \end{aligned} \quad (2.2)$$

introducido en 1.5.

La condición de estacionariedad está dada por ser la solución del sistema

$$\begin{aligned} \nabla_z F(z_{est}) + \sum_{i=1}^q \mu_i \nabla_z g_i(z_{est}) + \sum_{j=1}^s \beta_j \nabla v_j(z_{est}) + \\ + \sum_{k=1}^{q_0} \alpha_k \nabla_z [\nabla_y f((z_{est}) + \sum_{j=1}^s \nabla_y v_j((z_{est}) \lambda_j)] &= 0 \\ \alpha \nabla_y v_j(x, y) - \gamma_j &= 0 \end{aligned}$$

donde, denotando

$$J_0^g = \{i : g_i(x, y) = 0\} \quad (2.3)$$

como los índices activos del líder,

$$J_0^v = \{j : v_j(x, y) = 0\},$$

los del seguidor y

$$J_0^\lambda = \{j : \lambda_j = 0\},$$

como los elementos de λ que son 0,

$$\begin{aligned} \beta_j &= 0, j \in \{1, \dots, s\} \setminus J_0^v, \\ \gamma_j &= 0, j \in \{1, \dots, s\} \setminus J_0^\lambda, \\ g_i(z_{est}) &\leq 0, \quad \mu_i \geq 0, \quad \mu_i g_i(z_{est}) = 0, i = 1, \dots, q. \end{aligned}$$

De esta forma

- $\alpha \in \mathbb{R}^m$: multiplicador asociado a la condición de KKT del problema de nivel inferior.
- $\mu_i \in \mathbb{R}^q$: multiplicador asociado a las restricciones $g_i(x, y)$ del líder.
- $\beta_j \in \mathbb{R}^s$: multiplicador asociado a las restricciones v_j del líder.
- $\gamma_j \in \mathbb{R}^s$: multiplicador asociado a las restricciones $\lambda_j \geq 0$.

Condiciones extra respecto al signo de β y γ se tendrán en cuenta en dependencia de si el usuario escoge que sea C, M, fuertemente estacionario o cumpla la condición $\alpha = 0$. Es por esto que el conjunto $\{1, \dots, s\}$ se particiona en

$$J_1^v = \{j | v_j(x, y) = 0 \wedge \lambda_j > 0\} \quad (2.4)$$

$$J_2^v = \{j | v_j(x, y) = 0 \wedge \lambda_j = 0\} \quad (2.5)$$

$$J_3^v = \{j | v_j(x, y) < 0 \wedge \lambda_j = 0\} \quad (2.6)$$

Cada índice activo tiene multiplicadores β_j y en dependencia del caso γ_j asociados. De esta forma $\beta_j = 0, j \in J_3^v, \gamma_j = 0, j \in J_1^v$. Las condiciones de signo de la clase de punto estacionario correspondiente se analizan para $\beta_j, \gamma_j = 0, j \in J_2^v$.

Usando estas notaciones, en la próxima sección se muestra como se genera el problema.

Generación del problema sobre Julia

A continuación se mostrarán los pasos a seguir para generar el problema deseado.

2.2.1. Datos de entrada

Lo primero que hace el usuario es escoger si $\alpha = 0$ o no. En el segundo caso se le solicita que introduzca el valor de dicho multiplicador. Luego se declaran las variables diferenciando las del líder x y las del seguidor y . Se chequea que la dimensión de α y de y coincidan.

Con las variables se introduce la función objetivo del líder $F(x, y)$ y las restricciones $g_1(x, y), \dots, g_q(x, y)$. En cada caso el usuario define si desea que sea activa o no y para las activas le introduce el correspondiente multiplicador μ_j . De esta forma se construye el conjunto $J_0^G = \{i | g_i(x, y) = 0\}$ y se logra que $g_i(x, y)\mu_i = 0$ para todo $i = 1, \dots, q$. Se le advierte al usuario que μ_j debe ser no negativo, dándole la posibilidad de cambiarlo de haber cometido un error.

De manera análoga se procede con la función objetivo del seguidor $f(x, y)$, sus restricciones $v_1(x, y), \dots, v_s(x, y)$ y sus multiplicadores λ no negativos. Sabiendo que al menos uno de ellos $v_j(x, y)\lambda_j = 0$, se construye la partición (J_1^v, J_2^v, J_3^v) del conjunto $\{1, \dots, s\}$ descrita en (2.4), (2.5), (2.6).

Finalmente se introducen el resto de los multiplicadores, a saber β y γ , de forma que cumplan la condición de la clase de estacionariedad correspondiente. Se consideran dos casos diferentes

- $\alpha = 0$: Se hace automáticamente $\gamma = 0$ y el usuario introduce el multiplicador $\beta_j, j = 1, \dots, s$ de forma tal que:

C-estacionario:

- si $j \in J_1^v \cup J_2^v$: β_j libre.
- si $j \in J_3^v$: $\beta_j = 0$.

M-estacionario:

- si $j \in J_1^v \cup J_2^v$: β_j libre.
- si $j \in J_3^v$: $\beta_j = 0$.

fuertemente-estacionario:

- si $j \in J_1^v$: β_j libre.
 - si $j \in J_2^v$: $\beta_j \geq 0$.
 - si $j \in J_3^v$: $\beta_j = 0$.
- $\alpha \neq 0$: si cada $j = 1, \dots, q$ se introduce el par de multiplicadores $\beta_j, \gamma_j, j = 1, \dots, s$ de forma tal que:

C-estacionario:

- si $j \in J_1^v$: β_j libre, $\gamma_j = 0$.
- si $j \in J_2^v$: $\beta_j \gamma_j \geq 0$.
- si $j \in J_3^v$: $\beta_j = 0, \gamma_j$ libre.

M-estacionario:

- si $j \in J_1^v$: β_j libre, $\gamma_j = 0$.
- si $j \in J_2^v$: $\beta_j, \gamma_j > 0$ o $\beta_j \gamma_j = 0$.
- si $j \in J_3^v$: $\beta_j = 0, \gamma_j$ libre.

fuertemente-estacionario:

- si $j \in J_1^v$: β_j libre, $\gamma_j = 0$.
- si $j \in J_2^v$: $\beta_j, \gamma_j \geq 0$.
- si $j \in J_3^v$: $\beta_j = 0, \gamma_j$ libre.

2.2.2. Modificación del Problema Original

Después de tener los datos de entrada necesarios se procede a la modificación del problema de entrada para que este sea estacionario del tipo requerido, z_{est} .

Primero en caso de que $\alpha \neq 0$ en los $v_j(x, y) \in J_0^v$ se procede a calcular el $\mathbf{b}_j$ de la siguiente forma:

- $v_j(x, y) \in J_1^v$ **2.4:**

$$\mathbf{b}_j = \frac{\alpha \cdot (-\nabla_y v_j(z_{est}))^T \cdot \alpha}{\|\alpha\|_2^2} \quad (2.7)$$

- $v_j(x, y) \in J_2^v \cup J_3^v$ **(2.5), (2.6)**

$$\mathbf{b}_j = \frac{\alpha \cdot ((-\nabla_y v_j(z_{est}))^T \cdot \alpha) + \gamma_j}{\|\alpha\|_2^2} \quad (2.8)$$

Si $\alpha = 0$, como $\gamma = 0$, se toma $\mathbf{b}_j = 0$

Después de realizado el cálculo del $\mathbf{b}_j$, se procede a modificar la función por una constante para lograr que sean activas en z_{est} aquellas que fueron escogidas por el usuario y que todas sean no positivas. Para ello se evalúa

$$\hat{v}_j = v_j(z_{est}) + (\mathbf{b}_j^T) \cdot (y_1, y_2, \dots, y_m).$$

Se consideran los siguientes casos

- Si $\hat{v}_j \neq 0$ y $j \in J_1^v \cup J_2^v$, entonces $c_j^v = -\hat{v}_j$.
- Si $\hat{v}_j \geq 0$ y $j \in J_3^v$, entonces c_j^v se obtiene generando un número aleatorio entre $-(\hat{v}_j + 0.01), -(\hat{v}_j + 1)$.
- En otro caso $c_j^v = 0$.

De ahí que las restricciones del nivel inferior tengan la forma

$$v_j^*(x, y) = v_j(x, y) + (\mathbf{b}_j^T) \cdot (y_1, y_2, \dots, y_m) + c_j^v \quad (2.9)$$

Con estas condiciones se logra que

$$\begin{aligned} \alpha \nabla_y v_j^*(z_{est}) &= 0, & j \in J_1^v, \\ \alpha \nabla_y v_j^*(z_{est}) - \gamma_j &= 0, & j \in J_2^v \cup J_3^v, \\ v_j^*(z_{est}) &= 0, & j \in J_1^v \cup J_2^v, \\ v_j^*(z_{est}) &< 0, & j \in J_3^v. \end{aligned}$$

De forma análoga se agregan constantes c_i^g a las restricciones del líder, teniéndose que para

$$g_i^*(x, y) = g_i(x, y) + c_i^g,$$

se cumple

$$\begin{aligned} g_i^*(z_{est}) &= 0, & j \in J_0^g, \\ g_i^*(z_{est}) &< 0, & j \in \{1, \dots, q\} \setminus J_0^g. \end{aligned}$$

Solo falta lograr el cumplimiento de las condiciones de KKT del problema del seguidor. Para ello se considera el sistema

$$\nabla_y f(z_{est}) + \sum_{j \in J_2^v} (\lambda_j \nabla_y v_j^*(z_{est})) + \mathbf{b} \mathbf{f} = \mathbf{0}. \quad (2.10)$$

KKT del problema del nivel inferior

De esta manera la función objetivo del seguidor es

$$f^*(x, y) = f(x, y) + \mathbf{b} \mathbf{f} y$$

Finalmente se toman las derivadas del sistema de KKT con respecto a (x, y) . Para ello se considera el sistema

$$\begin{aligned} \nabla_{xy} F(z_{est}) + \sum_{j \in J_0^g} (\mu_i \nabla_{xy} g(z_{est})) + [\nabla_{x,y} \nabla_y f(z_{est}) + \sum_{j \in J_1^v} \lambda_j \nabla_{xy} \nabla_y v_j^*(z_{est})] \alpha + \\ \sum_{j \in J_1^v \cup J_2^v} (\beta_j \nabla_{xy} v_j^*(z_{est}) + \mathbf{B} \mathbf{F}) = \mathbf{0}, \end{aligned}$$

y se obtiene

$$F^*(x, y) = F(x, y) + \mathbf{B} \mathbf{F}(x, y)^T.$$

De esta forma el problema de dos niveles

$$\begin{aligned} \min_x & F^*(x, y) \\ \text{sujeto a} & \\ g_i^*(x, y) & \leq 0, \quad i = 1, \dots, q, \\ y \in \operatorname{argmin}_y & \left\{ f^*(x, y) \mid v_j^*(x, y) \leq 0, \quad j = 1, \dots, q, \right\} \end{aligned} \quad (2.11)$$

cumple que el punto (z_{est}, λ_{est}) es un punto estacionario del modelo MPEC correspondiente.

La salida del generador son las funciones $F^*(x, y), f^*(x, y), g_1^*(x, y), \dots, g_q^*(x, y), v_1^*(x, y), \dots, v_s^*(x, y)$.

Implementación Algorítmica y Guía de Usuario

En la implementación computacional del generador propuesto en la sección anteriores se utilizó el lenguaje de programación **Julia**, ver JuliaLang 2025, dado a su versatilidad en expresiones y funciones matemáticas implementadas en su paquete base y sus bibliotecas externas como **BilevelJuMP**, ver Garcia et al. 2022, que permite resolver problemas binivel SLSF lineales y cuadráticos sin tener que transformar el problema original y **JuMP** Lubin et al. 2023 el cual, es una interfaz robusta para la optimización general, todo ello con unas excelentes prestaciones de cómputo. Por ello se brinda una guía de usuario para el uso del generador disponible en <https://fvsb.github.io/Tesis/>. La implementación algorítmica se ha llevado a cabo mediante la creación de una biblioteca de Julia llamada **ProblemGenerator**, con una sintaxis *JuMP-like* intuitiva. Primero debe tenerse un problema de optimización Binivel SLSF planteado como el 1.5, el punto que se desea que sea estacionario (z_{est}), los índices activos descritos anteriormente según el caso así como sus multiplicadores correspondientes, que cumplirán las propiedades del tipo de estacionariedad requerida y la opción de tener α igual a o distinto de cero.

Para una mayor facilidad de comprensión del uso de la biblioteca se utiliza este problema ejemplo:

$$\begin{aligned}
 & \min_{x_1, x_2} && x_1^2 y_1^2 y_2 + x_2 \\
 & \text{s.t.} && x_1 + y_2 - y_1 \leq 9, \\
 & && \min_{y_1, y_2} && x_2^2 y_1^2 y_2 + x_1 \\
 & && \text{s.t.} && x_1^2 y_1^2 + x_2 \leq 0.
 \end{aligned} \tag{2.12}$$

Se debe tener instalado en el entorno de desarrollo de Julia los siguientes módulos:

- Symbolics
- LinearAlgebra

Para la utilización de la biblioteca se procede a su importación de la siguiente forma:

```
1 using ProblemGenerator
```

Ejemplo de código 2.1: Importar el Módulo

Para comenzar debe llamarse a la función **GeneratorModel** para crear el modelo base inicial.

Para $\alpha = 0$

Para crear un modelo donde $\alpha = 0$:


```

1      # Crear el modelo base del generador alpha=0
2      model=GeneratorModel()
3

```

Ejemplo de código 2.2: Crear el modelo para $\alpha = 0$

Para $\alpha \neq 0$

Para crear un modelo donde $\alpha \neq 0$, se tiene que pasar como parámetro el valor del α , el cual será un vector de *Number*:

```

1      # Crear el modelo base del generador alpha!=0
2      alpha_vec=Vector::{Number}
3      model=GeneratorModel(alpha_vec)

```

Ejemplo de código 2.3: Crear el modelo para $\alpha \neq 0$

En caso que se requiera como entrada un **Problem**, se debe declarar de que nivel se trata de evaluar el **model** en las funciones **Upper** para el nivel superior y **Lower** para el inferior.

Para referirse al nivel superior

```

1      # Referirse nivel superior
2      Upper(model)

```

Ejemplo de código 2.4: Referirse al nivel superior

Para referirse al nivel inferior

```

1      # Referirse nivel inferior
2      Lower(model)

```

Ejemplo de código 2.5: Referirse al nivel inferior

Las variables deben de declararse bajo la siguiente macro **@myvariables**, la cual recibe una función **Upper** o **Lower** que recibe el modelo para las variables del nivel superior y las del nivel inferior respectivamente.

Introduccion de las variables del nivel superior

```

1      # Se declara en el nivel superior las variables x_1, x_2
2      @myvariables Upper(model) x_1, x_2

```

Ejemplo de código 2.6: Introducir las variables del nivel superior

Introducción de las variables del nivel inferior

```

1 # Se declara en el nivel inferior las variables y_1, y_2
2 @myvariables Lower(model) y_1,y_2

```

Ejemplo de código 2.7: Introducir las variables del nivel inferior

2.3.1. Declarar Funciones Objetivo

Para declarar las funciones objetivos debe asumirse que en ambos casos es un problema de minimización. Se utilizará la función **SetObjectiveFunction** que recibe un **Problem** y una expresión de **Num**.

Declaración de una función objetivo del nivel superior

Con:

$$\min(x_1^2 * y_1^2 * y_2) + x_2$$

```

1 # Declarar la funcion objetivo del nivel superior
2 # Min de ((x_1^2)*(y_1^2)*(y_2))+x_2
3 SetObjectiveFunction(Upper(model),((x_1^2)*(y_1^2)*(y_2))+x_2)

```

Ejemplo de código 2.8: Declarar una función objetivo del nivel superior

Declaración de una función objetivo del nivel inferior

Con:

$$\min(x_2^2 * y_1^2 * y_2) + x_1$$

```

1 # Declarar la funcion objetivo del nivel inferior
2 # Min de ((x_2^2)*(y_1^2)*(y_2))+x_1
3 SetObjectiveFunction(Lower(model),((x_2^2)*(y_1^2)*(y_2))+x_1)

```

Ejemplo de código 2.9: Declarar una función objetivo del nivel inferior.

2.3.2. Definición del conjunto de Índices activos

Antes de ilustrar como introducir las restricciones se va a explicar que los tipos de índices activos.

- **Ambos Niveles:**

- **Normal** si no es un índice activo.

- **Nivel Superior:**

- **J_0_g** Si es un índice como en (2.3).

- **Nivel Inferior:**

- **J_0_LP_v** Si es un índice como en 2.4.
- **J_0_L0_v** Si es un índice como en 2.5.
- **J_Ne_L0_v** Si es un índice como en 2.6.

Los **RestrictionSetType** deben expresar en código de esta forma:

```

1 Normal
2 J_0_g
3 J_0_LP_v
4 J_0_L0_v
5 J_Ne_L0_v

```

Ejemplo de código 2.10: Definir el conjunto de índice activo

Los **RestrictionSetType** son un *enum* de Julia.

Para declarar las restricciones se brindan dos funciones:

- **SetLeaderRestriction:** Para el nivel superior.

- **SetFollowerRestriction:** Para el nivel inferior.

- **Nivel Superior:**

Para declarar las restricciones del nivel superior debe por cada restricción llamarse a la función **SetLeaderRestriction** con:

- El modelo (**model**)
- La expresión de la restricción, del tipo **Num**
- El tipo de restricción, del tipo **RestrictionSetType**
- El valor de μ_i correspondiente, del tipo **Number**

Ejemplo para introducir la restricción:

$$x_1 + y_2 - y_1 \leq 9$$

Índice activo del tipo J_0_g 2.3

$$\mu_i = 0,3$$

```

1 # Ejemplo de restriccion del nivel superior
2 SetLeaderRestriction(model,x_1+y_2-y_1<=9,J_0_g,0.3)

```

Ejemplo de código 2.11: Introducir restricción del nivel superior

■ Nivel Inferior:

Para declarar las restricciones del nivel inferior debe por cada restricción llamarse a la función **SetFollowerRestriction** con:

- El modelo (**model**)
- La expresión de la restricción, del tipo **Num**
- El tipo de restricción, del tipo **RestrictionSetType**
- El valor de β_j correspondiente, del tipo **Number**
- El valor de λ_j correspondiente, del tipo **Number**
- El valor de γ_j correspondiente en caso de ser valor de entrada, del tipo **Number**

Ejemplo para introducir la restricción:

- $\lambda_j \neq 0$:

$$((x_1^2) * (y_1^2)) + x_2 \leq 0$$

Índice activo del tipo J_Ne_L0_v2,6

$$\beta_j = 0,1$$

$$\lambda_j = 0,5$$

```

1 # Para caso lamda_j=0
2 SetFollowerRestriction(model,((x_1^2)*(y_1^2))+x_2
3 <=0,J_Ne_L0_v,0.1,0.5,0)

```

- $\lambda_j = 0$:

Análogo al caso anterior pero con $\lambda_j = 0$, por lo que γ_j valor de entrada

$$\beta_j = 0,1, \quad \lambda_j = 0, \quad \gamma_j = 0,4$$

```

1 # Para caso lamda_j!=0
2 SetFollowerRestriction(model,((x_1^2)*(y_1^2))+x_2
3 <=0,J_Ne_L0_v,0.1,0,0.4)

```

2.3.3. Introducir el Punto

Ahora debe de introducirse el valor del punto que debe ser estacionario de la clase seleccionada. Debe de tenerse en cuenta que para todas las variables declaradas en ambos niveles debe de definirse el valor de la componente.

```
1 # Introducir el punto (1,1,1,1)
2 SetPoint(model,Dict(x_1=>1,x_2=>1,y_1=>1,y_2=>1))
```

Ejemplo de código 2.12: Introducir el punto (1,1,1,1)

2.3.4. Generar el Problema

Finalmente al tener todos los datos previos introducidos se llama al **CreateProblem** pasándole como parámetro el *model* y generará dicho problema imprimiendo en consola este.

```
1 # Llamar para generar el problema
2 problem=CreateProblem(model)
```

Ejemplo de código 2.13: Generar el problema

2.3.5. Ejemplo completo

Se muestra el ejemplo completo para el problema antes propuesto:

```
1 # Importar dependencias necesarias
2 using ProblemGenerator
3 # Crear el modelo base del generador alpha=0
4 model=GeneratorModel()
5 #Declaracion de variables
6 # Se declara en el nivel superior las variables x_1, x_2
7 @myvariables Upper(model) x_1, x_2
8 # Se declara en el nivel inferior las variables y_1, y_2
9 @myvariables Lower(model) y_1,y_2
10 # Declarar la funcion objetivo del nivel superior
11 # Min de ((x_1^2)*(y_1^2)*(y_2))+x_2
12 SetObjectiveFunction(Upper(model),((x_1^2)*(y_1^2)*(y_2))+x_2)
13 # Ejemplo de restriccion del nivel superior
14 SetLeaderRestriction(model,x_1+y_2-y_1>9,J_0_g,0.3)
15 # Declarar la funcion objetivo del nivel inferior
16 # Min de ((x_2^2)*(y_1^2)*(y_2))+x_1
17 SetObjectiveFunction(Lower(model),((x_2^2)*(y_1^2)*(y_2))+x_1)
18 # Para caso lamda_j!=0
19 SetFollowerRestriction(model,((x_1^2)*(y_1^2))+x_2<=0,J_Ne_L0_v,0.1,0,0.4)
20 # Introducir el punto (1,1,1,1)
```

```
21 SetPoint(model,Dict(x_1=>1,x_2=>1,y_1=>1,y_2=>1))  
22 # Llamar para generar el problema  
23 problem=CreateProblem(model)
```

Ejemplo de código 2.14: Script

API e Implementación Gráfica

La solución computacional integra una API REST desarrollada en Julia mediante HTTP.jl y una interfaz gráfica web implementada en Python con *Streamlit*. La arquitectura cliente-servidor aprovecha la eficiencia de Julia para los cálculos matemáticos en el backend mientras proporciona una interfaz de usuario accesible a través de Streamlit. La interfaz facilita la generación de problemas en distintos puntos estacionarios, automatizando la generación de multiplicadores λ_j , β_j , γ_j según el tipo de punto requerido. El sistema permite la generación automática de puntos aleatorios y del vector α dentro de intervalos configurables, establecidos por defecto en $[0,1]$, y ofrece la exportación de los problemas generados en formato Excel. La documentación técnica y guías de usuario se encuentran en desarrollo activo en los repositorios oficiales del proyecto.

Capítulo 3

Experimentación

En este capítulo se presenta la fase experimental basada en una selección de problemas de optimización binivel extraídos de la literatura especializada Zhou et al. 2018. El estudio abarca tres categorías fundamentales de problemas: Lineales, Cuadráticos y No Convexos. La metodología experimental se desarrollará mediante la utilización de las bibliotecas de Julia para la identificación de mínimos locales, seguida de la perturbación de estos puntos mediante la introducción de valores aleatorios. A partir de estas modificaciones, se procederá a la generación de problemas estacionarios clasificados en 4 tipos específicos: $\alpha = 0$, C , M y *fuerte*

Los problemas modificados serán posteriormente evaluados utilizando algoritmos tradicionales implementados en Julia, con el objetivo de analizar su eficacia en relación al valor de la función objetivo del nivel superior. Este proceso permitirá determinar el impacto de las modificaciones realizadas y la robustez de los algoritmos frente a las variaciones introducidas.

Para garantizar un análisis exhaustivo y representativo, se han seleccionado un total de quince problemas de optimización binivel, distribuidos equitativamente entre las tres categorías mencionadas, asignando cinco problemas a cada una de ellas. Los problemas correspondientes a las categorías Lineales y Cuadráticos han sido extraídos de Floudas 1999, mientras que los problemas No Convexos provienen de Zhou et al. 2018. Esta selección diversa permite una evaluación comprehensiva de las diferentes características y comportamientos de los problemas de optimización binivel bajo estudio.

Modelación de la experimentación

En esta sección se describe el proceso de experimentación, donde se toman las siguientes consideraciones. Todos los valores, han sido redondeados por exceso a dos cifras decimales. Los problemas escogidos para la experimentación son:

Tabla 3.1: Problemas Seleccionados

No Convexos	Lineales	Cuadráticos
MitsosBarton2006Ex312	ex9.1.1	ex9.2.1
MitsosBarton2006Ex313	ex9.1.2	ex9.2.2
MitsosBarton2006Ex314	ex9.1.8	ex9.2.3
MitsosBarton2006Ex323	ex9.1.9	ex9.2.4
MorganPatrone2006a	ex9.1.10	ex9.2.5

Inicialmente se computan óptimos de los problemas con los paquetes convencionales de Julia cuya forma de resolver dependerá de la clase del problema. A continuación definiremos cada tipo de clase y como se obtienen los óptimos.

■ **Problemas Lineales y Cuadráticos :**

Un problema binivel lineal es aquel en el cual los problemas del nivel superior e inferior son lineales, análogamente se define el problema cuadrático.

Con dicho problema se introducen los datos en la interfaz de **BilevelJuMP**, ver Garcia et al. 2022, con el cual se utilizan 3 técnicas entre las ofrecidas por esta:

- **Big-M :** Con el optimizador High-Performance Solver for Linear Programming (HiGHS) y los valores primal big M = 100, dual big M = 100.
- **SOS1 :** Con el optimizador Solving Constraint Integer Programs (SCIP).
- **ProductMode :** Con el optimizador Interior Point Optimizer (Ipopt).

Cada uno de los resultados de evaluar el problema en cada forma anterior se guardan en un formato *.xlsx* donde por cada optimizador se guardan los parámetros:

- Estatus del Primal, el cual define si es un punto factible o no.
- Estatus de la Finalización, si terminó porque encontró un óptimo o se estancó en un óptimo local.
- Valor de la función objetivo del nivel superior.
- El punto óptimo encontrado, en caso de ser hallado.
- El tiempo de ejecución.

Posteriormente se analizan los resultados de dichos métodos, se selecciona el de mejor evaluación de la función objetivo.

■ **Problemas No Convexos :**

Al **BilevelJuMP** no contar con soporte para esta clase de problemas se utiliza **JuMP**, ver Lubin et al. 2023. Por ello se utiliza la reformulación KKT como la de 1.5 y se procede a utilizar la interfaz brindada por este. Para el caso de las restricciones de complementariedad se utiliza **Complementarity**, ver Chkwon 2025, con el optimizador Ipopt y análogo al caso anterior se extraen los mismos datos.

Generación de los problemas

Para la generación de los problemas, se toman los problemas seleccionados anteriormente con su punto óptimo calculado por las bibliotecas antes dichas, para ello mediante código de Julia se procede a generar los 4 tipos de problemas modificados: con $\alpha = 0$, *C-Estacionario*, *M-Estacionario* y *Fuertemente Estacionario*, mediante código de Julia para dichos experimentos. Cada experimento se realizará con semillas para poder replicarlos, cuya asignación se muestran en las tablas siguientes:

Tabla 3.2: Semillas utilizadas en problemas Lineales

No Convexos	Valor de la semilla para generar los problemas	Valor de la semilla para hallar los óptimos de cada problema modificado
ex9.1.1	20	6
ex9.1.2	2	8
ex9.1.8	3	9
ex9.1.9	4	10
ex9.1.10	5	7

Tabla 3.3: Semillas utilizadas en problemas Cuadráticos

No Convexos	Valor de la semilla para generar los problemas	Valor de la semilla para hallar los óptimos de cada problema modificado
ex9.2.1	11	11
ex9.2.2	12	12
ex9.2.3	13	13
ex9.2.4	14	14
ex9.2.5	15	15

Tabla 3.4: Semillas utilizadas en problemas No Convexos

No Convexos	Valor de la semilla para generar los problemas	Valor de la semilla para hallar los óptimos de cada problema modificado
MitsosBarton2006Ex312	6	1
MitsosBarton2006Ex313	7	2
MitsosBarton2006Ex314	9	3
MitsosBarton2006Ex323	10	4
MorganPatrone2006a	11	5

Metodología de Generación y Experimentación

La experimentación consta de 3 partes fundamentales: generación del problema perturbado (para cumplir con cierta clase de estacionariedad en un punto dado), cómputo de óptimos del problema, análisis de los resultados.

Para la generación de los diferentes tipos de problemas perturbados utiliza semillas para la generación de los valores aleatorios para una replicación del mismo. La implementación sigue una sintaxis similar a la descrita en el capítulo anterior, dado que esta nueva implementación incorpora y extiende la anterior, pero permite que con solo introducir un problema base se puedan generar los 4 tipos de problemas perturbados. El proceso de modificación de los puntos comienza con el problema original de entrada y el punto obtenido previamente. A cada componente de este punto se le añade un valor aleatorio en el intervalo $[5e - 10, 5]$, obteniendo así el punto modificado z_0^* . Para los casos donde $\alpha \neq 0$, se genera un vector aleatorio con componentes en el intervalo $[3e - 10, 3]$. En cuanto a los conjuntos de índices activos de las v_j s, estos se distribuyen siguiendo una proporción específica: 1/2 corresponde al tipo J_1^v (2.5), mientras que el 1/4 restante se divide entre los tipos 2.4 y 2.6.

La selección de los multiplicadores λ_j , β_j y γ_j se realiza mediante la generación de valores aleatorios en el intervalo $[1e - 2, 10]$ cuando estos no deben ser 0. En los casos donde existen múltiples combinaciones posibles de multiplicadores con respecto a su igualdad a 0, se utiliza una distribución uniforme discreta para la selección aleatoria. Cada problema generado se almacena en un archivo *.xlsx* siguiendo la nomenclatura: *(nombre del problema)_(Tipo de punto estacionario)(generator)_alpha_((non_zero) si $\alpha \neq 0$ y (zero) si $\alpha = 0$).xlsx*. Este archivo contiene la información completa del problema, incluyendo las expresiones de las funciones objetivo de ambos niveles con sus evaluaciones en el punto, las restricciones de ambos niveles con sus respectivos multiplicadores y tipos de índices activos, el punto z_0^* , los vectores **bf** y **BF**, y el multiplicador α .

La compilación y procesamiento posterior se realiza mediante un script de Python

que utiliza las dependencias *pandas* y *sympy*. Este script procesa la información contenida en los archivos *.xlsx* y genera scripts *.jl* con el módulo de experimentación para la obtención de valores óptimos. La ejecución de estos scripts se gestiona mediante *subprocess*, y los resultados se recopilan para identificar los casos más relevantes y generar tanto el código latex de cada problema modificado como las tablas de resultados correspondientes.

Los resultados seleccionados se presentan en tablas estructuradas que incluyen información fundamental para cada problema, hay una por cada tipo de problema y tipo de punto lo que hace una total de 12. Estas tablas contienen el nombre del problema original que fue modificado para alcanzar la estacionariedad de la clase deseada, el punto al cual se forzó ser estacionario de la clase requerida (asegurando que no pertenezca a un subconjunto de esta), la evaluación de la función objetivo en el punto estacionario, el punto óptimo identificado por los algoritmos de Julia, la evaluación de la función objetivo en este óptimo, el método seleccionado para su obtención y el tiempo requerido para su ejecución.

3.2.1. Problemas Lineales

$\alpha = 0$

En todos problemas lineales cuyos multiplicador $\alpha = 0$, existió solución óptima por parte de los métodos de Julia, en 3 de ellos el valor del óptimo fue capaz de mejorar el valor de la función objetivo en el punto estacionario inicial. Big-M fue el que obtuvo el de menor valor en cada uno, que coincidió con SOS1 en los 3 casos que este ofreció respuesta. Además hubo 2 problemas cuya evaluación inicial coincidió con la del óptimo (ambos iguales a 0). Por otro lado, ProductMode obtuvo puntos infactibles en 3 ocasiones.

C-Estacionario

En los problemas con un punto C-Estacionario conocido hubo 2 problemas para los cuales Julia no dio óptimo. En los 3 restantes la evaluación de las soluciones obtenidas por los métodos y los puntos estacionarios iniciales fueron cercanas, en estos casos SOS1 logró dar solución en todos al igual que ProductMode, aunque este devolvió óptimos locales. Big-M no logró ofrecer solución en ninguno de los 5 casos. Cabe destacar que Big-M puede tener dificultades en ciertos problemas por los valores de asociados al valor de las cotas de las cotas M_p y M_d .

M-Estacionario

Para los problemas con este tipo de punto estacionario conocido solo hubo 2 casos en los que se halló óptimo. En ambos los valores de la evaluación de punto inicial y del óptimo fueron casi iguales. En el primero solo Big-M dio solución mientras que en el segundo los 3 métodos devolvieron resultados, siendo el de ProductMode un óptimo local. El método más rápido en calcular la solución fue Big-M.

Fuertemente Estacionario

Análogo al tipo de punto anterior, se obtuvieron óptimos en 2 problemas cuyos valores de la evaluación de punto estacionario inicial y del óptimo fueron casi idénticos. En este caso en ambos problemas los 3 métodos obtuvieron solución en ambos casos. Además resultados de ProductMode también fueron óptimos locales.

3.2.2. Problemas Cuadráticos

$$\alpha = 0$$

En el caso en que los puntos estacionarios tengan multiplicador $\alpha = 0$, existió al menos un método que fue capaz de hallar la solución óptima. El método Big-M solo funcionó en un caso y calculó una solución con peor evaluación de la función objetivo respecto al punto estacionario inicial. Mientras ProductMode y SOS1 hallaron soluciones en todos los casos, solo en uno logró mejorar el punto inicial y en los otros dos se propuso una solución peor.

C-Estacionario

Como en el caso anterior al menos un algoritmo fue capaz de resolver los modelos. El método Big-M solo funcionó en un problema donde mejoró con respecto a la evaluación del punto estacionario inicial, al igual que ProductMode y SOS1. Además en 2 problemas los métodos hallaron puntos con peor evaluación de la función objetivo que en el punto estacionario inicial. En otros 2 modelos se obtuvieron óptimos cercanos a este.

M-Estacionario

Para este tipo de puntos en todos los problemas el criterio de parada fue porque se había alcanzado la optimalidad en al menos un algoritmo. Big-M solo funcionó en un problema, donde halló una solución cercana al punto estacionario inicial, siendo el más rápido en este problema. Solo en 3 casos se logró un óptimo con evaluación similar al punto estacionario inicial y cercano a este. ProductMode solo obtuvo óptimos

locales. SOS1 obtuvo soluciones de forma más rápida en dos problemas mientras que ProductMode en el resto. En dos casos la solución obtenida es peor que el punto propuesto.

Fuertemente Estacionario

Hubo un problema que ningún enfoque logro resolver. En cuanto al resto Big-M solo dio solución en un problema, construido usando el ex9.2.3 como base. Los métodos ProductMode y SOS1 solo lograron hallar tres puntos factibles que eran puntos óptimos locales y en dos casos logró mejorar la solución inicial, pero con tiempos similares. Hubo un problema que ningún método logró resolver.

3.2.3. Problemas No Convexos

En este caso solo se resuelve la reformulación KKT con el paquete Ipopt, ya que BilevelJump solo resuelve problemas convexos. Los resultados son muy similares. En ningún caso logro mejorar el punto inicial hallado. Para $\alpha = 0$ esto sucedió en todos los casos excepto en el problema MitsosBarton2006Ex323 en que el algoritmo falló. Para problemas con puntos C-estacionarios conocidos generados, en un caso coincidieron y en los problemas MitsosBarton2006Ex313 y MitsosBarton2006Ex314 las soluciones halladas por Julia fueron peores que la ya conocida. Vale destacar que el valor de las variables del nivel superior son iguales al punto estacionario inicial. En el resto la soluciones el algoritmo no halló solución factible. Para puntos M-estacionarios, solo se obtuvieron soluciones de los problemas MitsosBarton2006Ex313 y MitsosBarton2006Ex314, obteniendo en ambos casos que el valor óptimo hallado por Julia arrojó peores resultados en comparación con el valor del punto estacionario inicial. En la clase de fuertemente estacionarios, excepto en el problema MitsosBarton2006Ex323, se obtuvieron puntos óptimos. Solo mejoran levemente su valor en MitsosBarton2006Ex312. Además, en todos los casos, los valores de las variables del nivel superior coinciden.

3.2.4. Análisis de los resultados

El análisis de los experimentos realizados en este capítulo permite observar distintos comportamientos según el tipo de problema analizado. En los problemas lineales, se evidenció que, a medida que se imponen condiciones más fuertes en los puntos, es más común que los métodos encuentren óptimos en la vecindad del punto original. Sin embargo, estas mismas condiciones también incrementan la posibilidad de que los algoritmos no logren encontrar una solución óptima y terminen en puntos infactibles. En particular, el método ProductMode fue el que tuvo como salida un punto infac-

tible más veces, seguido por SOS1. Además en ProductMode los todos los óptimos hallados fueron óptimos locales. Un comportamiento a destacar es que en caso que ProductMode y SOS1 devuelvan un óptimo es común que la evaluación de los estos fuera cercana. Por otra parte Big-M tuvo mayormente problemas asociados al valor de las cotas M_p y M_d . Cabe destacar que en ningún caso el valor objetivo fue peor que el valor inicial estacionario.

En los problemas cuadráticos, excepto en el caso fuertemente estacionario no se encontraron dificultades en los algoritmos para encontrar un punto óptimo. En ellos existe una mejora entre los tipos de puntos con condiciones más fuertes respecto a que el óptimo hallado por Julia se encuentre en una vecindad del punto estacionario inicial. Continuó la tendencia de ProductMode y SOS1 que si ambos encontrarán un óptimo la evaluación de este similar en métodos. También en estos Big-M solo pudo hallar puntos estacionarios en el problema ex9.2.3. En el caso de los problemas no convexos, los resultados sugieren que, cuanto más fuertes son las condiciones del punto, los algoritmos no encuentran puntos óptimos mejores que el estacionario inicial. Esto parece deberse a que, por la propia reformulación del problema, es más probable que no se pueda optimizar correctamente el problema del nivel inferior. Además, la tendencia indica que, mientras más fuertes sean las condiciones del punto, es más probable que las variables del nivel superior coincidan en el punto óptimo hallado por Julia y el punto estacionario inicial.

En conclusión, los resultados de los experimentos muestran como con este tipo de problemas se puede estudiar la fortaleza del desempeño de los algoritmos de optimización para problemas MPEC resultantes de la aplicación de enfoque KKT al problemas binivel.

Conclusiones

En esta tesis se desarrolló un algoritmo para la generación de problemas de optimización binivel con características específicas de estacionariedad en puntos determinados. El mismo tiene la capacidad de modificar problemas originales para garantizar la factibilidad y estacionariedad en puntos dados, aprovechando las capacidades del lenguaje de programación Julia, que destaca por su alto rendimiento computacional y sintaxis adaptada a problemas de optimización.

La experimentación se llevó a cabo sobre tres categorías fundamentales de problemas: lineales, cuadráticos y no convexos. El proceso experimental comenzó con la obtención de puntos mínimos utilizando bibliotecas establecidas como BilevelJuMP y JuMP. Posteriormente, se generaron problemas estacionarios mediante la adición de componentes aleatorias a estos puntos para cada clase de problema definida.

Los problemas modificados fueron sometidos nuevamente a algoritmos tradicionales implementados en Julia para evaluar su efectividad frente a los puntos estacionarios generados. Se realizó un análisis comparativo destacando los casos más relevantes de cada categoría de puntos estacionarios, considerando la evaluación de la función objetivo del nivel superior en el punto donde se garantizó la estacionariedad, contrastándola con los resultados obtenidos por las bibliotecas convencionales.

Si bien la experimentación debe ser mayor, se observan las dificultades en la solución algorítmica. Hubo varios problemas que no fueron solubles por ninguno de los algoritmos de Julia, inclusive dieron soluciones infactibles, si bien se sabían por la forma de generación que tenían al menos el punto estacionario generado como punto factible. Esto demuestra la dificultad estructural de los problemas de optimización de dos niveles y la necesidad de trabajar en algoritmos para su solución.

Como resultado de la investigación realizada, se han identificado varias líneas de trabajo futuro que permitirían expandir y mejorar los resultados obtenidos. En primer lugar, se recomienda ampliar el alcance de la experimentación numérica para incluir una mayor diversidad de problemas de optimización binivel. Esta expansión permitiría validar la robustez y versatilidad del algoritmo propuesto en diferentes contextos y escenarios de aplicación.

En segunda instancia, se sugiere profundizar en la investigación sobre la implementación del algoritmo desarrollado como criterio de parada en nuevos métodos de op-

timización binivel. Esta línea de investigación podría contribuir significativamente al desarrollo de algoritmos más eficientes y confiables, mejorando la capacidad de detectar y verificar puntos estacionarios durante el proceso de optimización.

Finalmente, se propone el desarrollo de una interfaz gráfica más intuitiva y funcional que facilite la generación automática de puntos según el tipo de estacionariedad requerida. Esta mejora en la usabilidad del software permitiría que usuarios con diferentes niveles de experiencia puedan aprovechar las capacidades del algoritmo de manera más efectiva, ampliando así su aplicabilidad práctica en diversos campos de estudio.

Bibliografía

- Aussel, D., Bendotti, P., & Pistek, M. (2017). Nash equilibrium in a pay-as-bid electricity market Part 2 - best response of a producer. *Optimization*, 66, 1027-1053. <https://api.semanticscholar.org/CorpusID:18648572> (vid. pág. 2).
- Aussel, D., Cervinka, M., & Marechal, M. (2016). Deregulated electricity markets with thermal losses and production bounds: models and optimality conditions. *RAIRO Oper. Res.*, 50, 19-38. <https://api.semanticscholar.org/CorpusID:41625879> (vid. pág. 1).
- Bard, J. F. (1991). Some properties of the bilevel programming problem. *Journal of Optimization Theory and Applications*, 68(2), 371-378. <https://doi.org/10.1007/BF00941574> (vid. pág. 3).
- Bhavsar, N., & Verma, M. (2021). A subsidy policy to managing hazmat risk in railroad transportation network. *Eur. J. Oper. Res.*, 300, 633-646. <https://api.semanticscholar.org/CorpusID:238706367> (vid. pág. 2).
- Bouza-Allende, G., Aussel, D., Dempe, S., & Lepaul, S. (2021). Genericity Analysis of Multi-Leader-Disjoint-Followers Game. *SIAM J. Optim.*, 31, 2055-2079. <https://api.semanticscholar.org/CorpusID:238717899> (vid. pág. 3).
- Bouza-Allende, G., & Still, G. (2012). Solving bilevel programs with the KKT-approach. *Mathematical Programming*, 138, 309-332. <https://api.semanticscholar.org/CorpusID:18500519> (vid. pág. 5).
- Caselli, G., Iori, M., & Ljubić, I. (2024). Bilevel optimization with sustainability perspective: a survey on applications. <https://api.semanticscholar.org/CorpusID:270379993> (vid. pág. 3).
- Cerulli, M. (2021, diciembre). *Bilevel optimization and applications* [Tesis doctoral]. (Vid. págs. 3, 4).
- Chkwon. (2025). Complementarity.jl. <https://github.com/chkwon/Complementarity.jl> (vid. págs. 16, 36).
- Dempe, S., & Freiberg, T. (2003). Annotated Bibliography on Bilevel Programming and Mathematical Programs with Equilibrium Constraints. *Optimization*, 52. <https://doi.org/10.1080/0233193031000149894> (vid. pág. 3).
- Dempe, S., & Zemkoho, A. (2020a). *Bilevel Optimization: Advances and Next Challenges*. (Vid. págs. 1, 3, 4, 7-9).

- Dempe, S., & Zemkoho, A. (2020b). *Bilevel Optimization: Advances and Next Challenges*. (Vid. pág. 2).
- Flegel, M. L., & Kanzow, C. (2003). A Fritz John Approach to First Order Optimality Conditions for Mathematical Programs with Equilibrium Constraints. *Optimization*, 52, 277-286. <https://api.semanticscholar.org/CorpusID:26882450> (vid. págs. 5, 9, 10).
- Floudas, C. A. (1999). Handbook of Test Problems in Local and Global Optimization. <https://api.semanticscholar.org/CorpusID:117898119> (vid. pág. 34).
- Floudas, C. A., & Pardalos, P. M. (1990). A Collection of Test Problems for Constrained Global Optimization Algorithms. *Lecture Notes in Computer Science*. <https://api.semanticscholar.org/CorpusID:139191> (vid. pág. 3).
- Garcia, J. D., Bodin, G., & Street, A. (2022). BilevelJuMP.jl: Modeling and Solving Bilevel Optimization in Julia. *ArXiv, abs/2205.02307*. <https://api.semanticscholar.org/CorpusID:248524800> (vid. págs. 12, 13, 19, 27, 35).
- Gowda, S., Ma, Y., Cheli, A., Gwozdz, M., Shah, V. B., Edelman, A., & Rackauckas, C. (2021). High-performance symbolic-numerics via multiple dispatch. *arXiv preprint arXiv:2105.03949*. <https://symbolics.juliasymbolics.org/stable/> (vid. pág. 20).
- Gu, H., Li, Y., Yu, J., Wu, C., Song, T., & Xu, J. (2020). Bi-level optimal low-carbon economic dispatch for an industrial park with consideration of multi-energy price incentives. *Applied Energy*, 262, 114276. <https://api.semanticscholar.org/CorpusID:213998633> (vid. pág. 2).
- Jeroslow, R. G. (1985). The polynomial hierarchy and a simple model for competitive analysis. *Mathematical Programming*, 32, 146-164. <https://api.semanticscholar.org/CorpusID:39987722> (vid. pág. 3).
- JuliaLang. (2025). *Julia Documentation*. <https://docs.julialang.org/en/v1/> (vid. pág. 27).
- Language, T. J. (2025). *LinearAlgebra - Julia Documentation* [Accessed: 2025-02-09]. <https://docs.julialang.org/en/v1/stdlib/LinearAlgebra/> (vid. pág. 20).
- Lubin, M., Dowson, O., Dias Garcia, J., Huchette, J., Legat, B., & Vielma, J. P. (2023). JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*. <https://doi.org/10.1007/s12532-023-00239-3> (vid. págs. 18, 27, 36).
- Ramos, M. A., Boix, M., Aussel, D., Montastruc, L., & Domenech, S. (2016). Water integration in eco-industrial parks using a multi-leader-follower approach. *Comput. Chem. Eng.*, 87, 190-207. <https://api.semanticscholar.org/CorpusID:26463725> (vid. pág. 2).
- Ramos, M. A., Rocafull, M., Boix, M., Aussel, D., Montastruc, L., & Domenech, S. (2018). Utility network optimization in eco-industrial parks by a multi-

- leader follower game methodology. *Comput. Chem. Eng.*, 112, 132-153. <https://api.semanticscholar.org/CorpusID:4003323> (vid. pág. 2).
- Schmidt, M., & Beck, Y. (2021). A Gentle and Incomplete Introduction to Bilevel Optimization. <https://www.lamsade.dauphine.fr/poc/sites/default/files/bilevel-optimization.pdf> (vid. pág. 8).
- Scholtes, S. (2001). Convergence properties of a regularization scheme for mathematical programs with complementarity constraints. *SIAM Journal on Optimization*, 11(4), 918-936 (vid. pág. 13).
- Siddiqui, S., & Gabriel, S. (2012). An SOS1-Based Approach for Solving MPECs with a Natural Gas Market Application. *Networks and Spatial Economics*, 13. <https://doi.org/10.1007/s11067-012-9178-y> (vid. págs. 4, 13).
- Sinha, A., Malo, P., & Deb, K. (2017). A Review on Bilevel Optimization: From Classical to Evolutionary Approaches and Applications. *IEEE Transactions on Evolutionary Computation*, 22, 276-295. <https://api.semanticscholar.org/CorpusID:4626744> (vid. págs. 3, 4, 8).
- Zhou, S., Zemkoho, A. B., & Tin, A. (2018). BOLIB: Bilevel Optimization LIBrary of Test Problems. *Bilevel Optimization*. <https://api.semanticscholar.org/CorpusID:119636540> (vid. pág. 34).

Anexos

Tablas de resultados

Tabla 3.5: Resultados de los problemas lineales del tipo Alpha-Zero

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
en0.1.1	(6.05, 6.85, 3.10.)	18.59	(6.13, 6.85, -173.65.)	-1820.0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.000467796639699995
en0.1.1	(6.05, 6.85, 3.10.)	18.59	(6.13, 6.85, -1.39e+19.)	-1.238428244918673e+20	ProductMode	NEARLY-FEASIBLE-POINT	ALMOST-LOCALLY-SOLVED	0.0182166562836365
en0.1.10	(51.55, 18.10, 102.20, 0.20.)	-0.48	(1.65, 34.73, 102.20, 100.00.)	240.0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0005916897039552061
en0.1.10	(51.55, 18.10, 102.20, 0.20.)	-0.48	(-1.01e+20, 3.37e+19, 676781.00, 2.02e+20.)	-4.8528604996887255e+20	ProductMode	INFEASIBLE-POINT	NORM-LIMIT	0.09548243866603773
en0.1.2	(7.00, 8.95.)	0.0	(7.00, 8.95.)	0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.000595508938271405
en0.1.2	(7.00, 8.95.)	0.0	(7.00, 8.95.)	0	SOS1	FEASIBLE-POINT	OPTIMAL	0.003649123239766818
en0.1.2	(7.00, 8.95.)	0.0	(7.00, 8.95.)	0	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.012145625483009709
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(-200.00, 0.00, -203.20, 208.25.)	0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0006034477299367704
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(5.14, 1.30, 5.82, 0.00.)	0	SOS1	FEASIBLE-POINT	OPTIMAL	0.003330911396261476
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(-3837804009426.00, 3837804592535.52, 7675609168177.35, 15351218367031.80.)	0	ProductMode	INFEASIBLE-POINT	OTHER-ERROR	0.0392210034453125
en0.1.9	(4.78, 6.17.)	3.98	(8.61, -1.48.)	-2.9	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0004643262269
en0.1.9	(4.78, 6.17.)	3.98	(8.61, -1.48.)	-2.9	SOS1	FEASIBLE-POINT	OPTIMAL	0.00512388362000001
en0.1.9	(4.78, 6.17.)	3.98	(4.79, 6.17.)	3.98	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.00962869485549133

Tabla 3.6: Resultados de los problemas lineales del tipo C-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
en0.1.1	(6.05, 6.85, 3.10.)	71.95	(0.00, 0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.000468240455
en0.1.1	(6.05, 6.85, 3.10.)	71.95	(6.25, 10.36, 6.60.)	-27.94	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.01053818123628692
en0.1.10	(51.55, 18.10, 102.20, 0.20.)	0.0	(50.81, 17.80, 126.86, 163.97.)	0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0006909111494300806
en0.1.10	(51.55, 18.10, 102.20, 0.20.)	0.0	(53.73, 19.99, 0.00, -664.87.)	0	SOS1	FEASIBLE-POINT	OPTIMAL	0.003464193543372658
en0.1.10	(51.55, 18.10, 102.20, 0.20.)	0.0	(50.81, 17.80, 126.75, 163.24.)	0	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0068813606955922865
en0.1.2	(7.00, 8.95.)	-533.33	(7.00, 8.95.)	-533.62	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0003651432419000004
en0.1.2	(7.00, 8.95.)	-533.33	(7.00, 8.95.)	-533.62	SOS1	FEASIBLE-POINT	OPTIMAL	0.0028385706809463617
en0.1.2	(7.00, 8.95.)	-533.33	(7.00, 8.95.)	-533.62	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.011949152739234449
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(0.00, 0.00, 0.00, 0.00.)	0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0004503267894
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(4.79, 1.52, -4.91, 0.00.)	0	SOS1	FEASIBLE-POINT	OPTIMAL	0.003091181772136223
en0.1.8	(4.50, 1.60, 6.10, 1.55.)	0.0	(157622.80, -157616.49, -204903418.75, -19284423.52.)	0	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.006741560248313091
en0.1.9	(4.78, 6.17.)	-359.91	(0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0005221321025921026
en0.1.9	(4.78, 6.17.)	-359.91	(4.78, 6.68.)	-385.79	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.005467831646608316

Tabla 3.7: Resultados de los problemas lineales del tipo M-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
en9.1.1	(6.05, 6.85, 3.10.)	-350.25	(0.00, 0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.000633438676526839
en9.1.1	(6.05, 6.85, 3.10.)	-350.25	(6.10, 2.56, 0.19.)	-353.19	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.009341708173831776
en9.1.10	(51.55, 18.10, 102.20, 0.20.)	256.63	(-561.59, -215.62, -247.16, -207.74.)	255.12	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0007100866538461538
en9.1.10	(51.55, 18.10, 102.20, 0.20.)	256.63	(-405840425249201.62, -15408567065387.44, -231326563895116.47, -137084383270963.31.)	-440622926211.5	ProductMode	INFEASIBLE-POINT	ITERATION-LIMIT	0.659487631
en9.1.2	(7.00, 8.95.)	-42.7	(0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0004965854924447435
en9.1.2	(7.00, 8.95.)	-42.7	(0.31, 0.13.)	-1.87	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.00501177204321608
en9.1.8	(4.50, 1.60, 6.10, 1.55.)	-17.69	(0.00, 0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0004395224573000003
en9.1.8	(4.50, 1.60, 6.10, 1.55.)	-17.69	(-0.14, 0.43, -0.13, 0.12.)	-0.83	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.004709428563097514
en9.1.9	(4.78, 6.17.)	0.0	(4.78, 6.16.)	0	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0004206910923
en9.1.9	(4.78, 6.17.)	0.0	(4.78, 6.16.)	0	SOS1	FEASIBLE-POINT	OPTIMAL	0.00354871923541963
en9.1.9	(4.78, 6.17.)	0.0	(-99475.59, -82209.03.)	0	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.007393484697407407

Tabla 3.8: Resultados de los problemas lineales del tipo Fuertemente-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
en9.1.1	(6.05, 6.85, 3.10.)	-195.55	(6.14, 6.84, 3.11.)	-195.39	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.001778082932761088
en9.1.1	(6.05, 6.85, 3.10.)	-195.55	(6.14, 6.84, 3.11.)	-195.39	SOS1	FEASIBLE-POINT	OPTIMAL	0.005636886166854566
en9.1.1	(6.05, 6.85, 3.10.)	-195.55	(6.14, 6.84, 3.11.)	-195.39	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.00896528261005386
en9.1.10	(51.55, 18.10, 102.20, 0.20.)	-17921.11	(0.00, 0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0005715390879475077
en9.1.10	(51.55, 18.10, 102.20, 0.20.)	-17921.11	(-151.11, 3.00, 39.70, -20.88.)	-5686.3	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.010776944681034482
en9.1.2	(7.00, 8.95.)	-957.15	(0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.0006340764801227152
en9.1.2	(7.00, 8.95.)	-957.15	(7.00, 2.20.)	-325.17	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.0073075757475146198
en9.1.8	(4.50, 1.60, 6.10, 1.55.)	-222.34	(0.00, 0.00, 0.00.)	0.0	High-BigM (100,100)	NO-SOLUTION	INFEASIBLE	0.000672642556480219
en9.1.8	(4.50, 1.60, 6.10, 1.55.)	-222.34	(-1180748808400.02, 46459818920.77, -1663916387913.37, 1630567269460.85.)	-82526799750.46	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.0518054399565288
en9.1.9	(4.78, 6.17.)	-542.78	(4.78, 6.17.)	-542.6	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0004550084491000005
en9.1.9	(4.78, 6.17.)	-542.78	(4.78, 6.17.)	-542.6	SOS1	FEASIBLE-POINT	OPTIMAL	0.004884637435972629
en9.1.9	(4.78, 6.17.)	-542.78	(4.78, 6.17.)	-542.6	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.00781831330973479

%endresultstable

Tabla 3.9: Resultados de los problemas cuadráticos del tipo Alpha-Zero

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
en9.2.1	(1.85, 4.05.)	-106.29	(2.19, 5.31.)	-108.96	SOS1	FEASIBLE-POINT	OPTIMAL	0.01085754318478261
en9.2.1	(1.85, 4.05.)	-106.29	(2.19, 5.31.)	-108.96	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.007574700737878789
en9.2.2	(14.45, 14.40.)	-415.5	(14.45, 14.40.)	-415.5	SOS1	FEASIBLE-POINT	OPTIMAL	0.002536806859319452
en9.2.2	(14.45, 14.40.)	-415.5	(14.45, 14.40.)	-415.5	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0049392438734916
en9.2.3	(1.55, 2.70, -5.10, -8.65.)	8.98	(-3.85, 2.70, -5.10, -11.35.)	30.31	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0014682835419126327
en9.2.3	(1.55, 2.70, -5.10, -8.65.)	8.98	(-3.85, 2.70, -5.10, -11.35.)	30.31	SOS1	FEASIBLE-POINT	OPTIMAL	0.006388017097186701
en9.2.3	(1.55, 2.70, -5.10, -8.65.)	8.98	(-3.85, 2.70, -5.10, -11.35.)	30.31	ProductMode	NEARLY-FEASIBLE-POINT	ALMOST-LOCALLY-SOLVED	0.01848814341697417
en9.2.4	(4.25, 4.85, 4.70.)	-91.56	(0.15, 4.85, -4.70.)	43.8	SOS1	FEASIBLE-POINT	OPTIMAL	0.0032146947907276237
en9.2.4	(4.25, 4.85, 4.70.)	-91.56	(0.15, 4.85, -4.70.)	43.8	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.004554985538742023
en9.2.5	(4.75, 4.05.)	3.4	(4.75, 4.05.)	3.4	SOS1	FEASIBLE-POINT	OPTIMAL	0.017031251251904762
en9.2.5	(4.75, 4.05.)	3.4	(4.75, 4.05.)	3.4	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.00521551028914405

Tabla 3.10: Resultados de los problemas cuadráticos del tipo C-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
ex9.2.1	(1.85, 4.65,)	-17.97	(1.85, 2.39,)	16.72	SOS1	FEASIBLE-POINT	OPTIMAL	0.030418006612121212
ex9.2.1	(1.85, 4.65,)	-17.97	(1.85, 2.39,)	16.72	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.04917132575490196
ex9.2.2	(14.45, 14.40,)	-1327.11	(14.48, 14.40,)	-1327.01	SOS1	FEASIBLE-POINT	OPTIMAL	0.003636090177713037
ex9.2.2	(14.45, 14.40,)	-1327.11	(14.48, 14.40,)	-1327.01	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0051832394190871375
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	363.83	(-0.18, 49.63, 5.89, 19.45,)	-548.42	Highs-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0045629535242451965
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	363.83	(32.97, -5.86, 15.95, 13.31,)	-1201.53	SOS1	FEASIBLE-POINT	OPTIMAL	0.0051704202712215324
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	363.83	(-0.18, 49.63, 5.89, 19.45,)	-548.42	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.004764864257633587
ex9.2.4	(4.25, 4.85, 4.70,)	-33.88	(-4.22, -0.13, -4.09,)	11.48	SOS1	FEASIBLE-POINT	OPTIMAL	0.0038767353470178156
ex9.2.4	(4.25, 4.85, 4.70,)	-33.88	(-4.22, -0.13, -4.09,)	11.48	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.003738380746067416
ex9.2.5	(4.75, 4.05,)	-42.78	(4.43, 4.19,)	-42.9	SOS1	FEASIBLE-POINT	OPTIMAL	0.01199349312708322
ex9.2.5	(4.75, 4.05,)	-42.78	(4.43, 4.19,)	-42.9	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.003723685645786726

Tabla 3.11: Resultados de los problemas cuadráticos del tipo M-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
ex9.2.1	(1.85, 4.65,)	-41.57	(1.85, 4.61,)	-41.63	SOS1	FEASIBLE-POINT	OPTIMAL	0.003688455623059867
ex9.2.1	(1.85, 4.65,)	-41.57	(1.85, 4.61,)	-41.63	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0146413623625731
ex9.2.2	(14.45, 14.40,)	-619.42	(11.02, 9.49,)	-488.71	SOS1	FEASIBLE-POINT	OPTIMAL	0.019986648072
ex9.2.2	(14.45, 14.40,)	-619.42	(11.02, 9.49,)	-488.71	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0027599958350857778
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	405.85	(1.48, 2.65, -5.08, -8.66,)	404.57	Highs-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.001168006057492931
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	405.85	(1.48, 2.65, -5.08, -8.66,)	404.57	SOS1	FEASIBLE-POINT	OPTIMAL	0.0031234532765957446
ex9.2.3	(1.55, 2.70, -5.10, -8.65,)	405.85	(1.48, 2.65, -5.08, -8.66,)	404.57	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.00878768395606327
ex9.2.4	(4.25, 4.85, 4.70,)	-53.06	(-303.76, -14.82, -288.94,)	43860.69	SOS1	FEASIBLE-POINT	OPTIMAL	0.006881493973829201
ex9.2.4	(4.25, 4.85, 4.70,)	-53.06	(-303.76, -14.82, -288.94,)	43860.69	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.004288314234536082
ex9.2.5	(4.75, 4.05,)	-123.02	(4.73, 4.06,)	-123.3	SOS1	FEASIBLE-POINT	OPTIMAL	0.028344932163841808
ex9.2.5	(4.75, 4.05,)	-123.02	(4.73, 4.06,)	-123.3	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.0050364884899193555

Tabla 3.12: Resultados de los problemas cuadráticos del tipo Fuertemente-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
cd9.2.1	(1.85, 4.65,)	-379.69	(1.85, 4.65,)	-379.65	SOS1	FEASIBLE-POINT	OPTIMAL	0.008176077955810147
cd9.2.1	(1.85, 4.65,)	-379.69	(1.85, 4.65,)	-379.65	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.005971634445639188
cd9.2.2	(14.45, 14.40,)	-2976.34	(14.45, 14.40,)	-2976.34	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.006671206622162884
cd9.2.3	(1.55, 2.70, -5.10, -8.65,)	293.57	(-6.90, -7.39, 0.87, -12.87,)	111.17	High-BigM (100,100)	FEASIBLE-POINT	OPTIMAL	0.0013926462687237028
cd9.2.3	(1.55, 2.70, -5.10, -8.65,)	293.57	(-6.90, -7.39, 0.87, -12.87,)	111.17	SOS1	FEASIBLE-POINT	OPTIMAL	0.003255103333767927
cd9.2.3	(1.55, 2.70, -5.10, -8.65,)	293.57	(-6.90, -7.39, 0.87, -12.87,)	111.17	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.020168756826612906
cd9.2.4	(4.25, 4.85, 4.70,)	-23.29	(-46.77, 0.00, -76.20,)	3537.11	ProductMode	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.006134994812039312
cd9.2.5	(4.75, 4.05,)	-197.17	(-0.56, 4.83,)	-236.14	SOS1	FEASIBLE-POINT	OPTIMAL	0.007121795407988588
cd9.2.5	(4.75, 4.05,)	-197.17	(-0.56, 4.83,)	-236.14	ProductMode	FEASIBLE-POINT	LOCALLY-SOLVED	0.007361066086892489

Tabla 3.13: Resultados de los problemas No Convexo del tipo Alpha-Zero

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
MitsosBarton2006Esc312	(1.90, 0.15,)	0.92	(1.90, 0.15,)	0.91	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.03566443956834532
MitsosBarton2006Esc313	(1.05, 2.85,)	-0.5	(1.05, 2.85,)	-0.5	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.03757509242424242
MitsosBarton2006Esc314	(0.90, 2.70,)	-8.4	(0.90, 2.70,)	-8.4	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.0474979219047619
MitsosBarton2006Esc323	(1.94, 3.70,)	60.69	(-1.83, 3.70,)	17.53	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.045412499090909095
MorganPatrone2006a	(2.85, 4.45,)	3.27	(2.85, 4.45,)	3.27	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.044472833928571424

Tabla 3.14: Resultados de los problemas No Convexo del tipo C-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
MitsosBarton2006Esc312	(1.90, 0.15,)	1.27	(1.90, 0.15,)	1.25	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.022789461467889908
MitsosBarton2006Esc313	(1.05, 2.85,)	-6.15	(1.05, 1.10,)	-2.02	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.017918101449275362
MitsosBarton2006Esc314	(0.90, 2.70,)	-19.06	(0.90, 0.95,)	-8.66	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.018785589393939393
MitsosBarton2006Esc323	(1.94, 3.70,)	179.12	(1.94, 3.70,)	179.08	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.04463823539823009
MorganPatrone2006a	(2.85, 4.45,)	-1.8	(0.68, 0.00,)	-0.43	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.025443517435897435

Tabla 3.15: Resultados de los problemas No Convexo del tipo M-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
MitsosBarton2006Esc312	(1.90, 0.15,)	3.71	(1.90, -0.80,)	4.02	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.2315981090090908
MitsosBarton2006Esc313	(1.05, 2.85,)	-7.97	(1.05, 1.10,)	-2.52	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.01772621146053403
MitsosBarton2006Esc314	(0.90, 2.70,)	-12.1	(0.90, 0.95,)	-6.5	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.02129611379310345
MitsosBarton2006Esc323	(1.94, 3.70,)	9.32	(1.94, 3.50,)	9.69	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.048881390196078435
MorganPatrone2006a	(2.85, 4.45,)	-1.45	(2.85, -89783940.52,)	-1.45	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.08921844821428572

Tabla 3.16: Resultados de los problemas No Convexo del tipo Fuertemente-Estacionario

Nombre del problema	Punto estacionario	Valor objetivo del punto estacionario	Punto óptimo	Valor del punto óptimo	Método seleccionado	Estatus Primal	Estatus Terminación	Tiempo Promedio
MitsosBarton2006Esc312	(1.90, 0.15,)	2.72	(1.90, 0.00,)	2.26	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.023039427906976745
MitsosBarton2006Esc313	(1.05, 2.85,)	-7.89	(1.05, 1.10,)	-2.86	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.07522934179104478
MitsosBarton2006Esc314	(0.90, 2.70,)	-39.2	(0.90, 0.95,)	-15.75	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.015471115887850467
MitsosBarton2006Esc323	(1.94, 3.70,)	-421.1	(1.94, 3.64,)	-415.75	Reformulacion KKT	INFEASIBLE-POINT	LOCALLY-INFEASIBLE	0.05190893125
MorganPatrone2006a	(2.85, 4.45,)	-85.51	(2.85, 4.45,)	-85.56	Reformulacion KKT	FEASIBLE-POINT	LOCALLY-SOLVED	0.07928214603174603